

ДИСЦИПЛИНА	Технологии разработки серверных приложений
ИНСТИТУТ	ИПТИП
КАФЕДРА	Индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Методические указания к практическим занятиям
ПРЕПОДАВАТЕЛЬ	Дворецкий Артур Геннадьевич
СЕМЕСТР	4 семестр, 2025/2026 уч. год

Ссылка на материал:

https://github.com/dvOretsky/fastapi-tutorial/blob/main/FAPI4_Postman/FAPI4_Postman.md

Практическое занятие №4: Postman для тестирования API

Основное назначение **Postman** – облегчить разработку, тестирование и документирование **API**. С его помощью можно отправить данные в запросе и проверить полученный ответ. Также у него есть много других интересных возможностей. Можно например сохранять запросы в папки и коллекции, удобно параметризовывать запросы. Запускать коллекции с помощью **Collection Runner** и использовать их как автоматизированные тесты. **Postman** позволяет проектировать дизайн **API** и создавать на его основе **Mock-сервер**.

Тестировщики с помощью **Postman** могут отправлять **HTTP/HTTPS** запросы к сервисам и получать ответы от них. С помощью такого подхода можно протестировать бэкенд сервисы и убедиться, что они работают корректно. С помощью **Postman** можно выполнять запросы к различным типам **API**, таким как **REST**, **SOAP** и **GraphQL**.

Postman – это сейчас достаточно популярный инструмент, который использует в работе много людей. У него есть большое сообщество, в котором в случае необходимости можно будет найти ответы на появившиеся вопросы.

Для установки под **Windows**:

- Загрузите [последнюю версию Postman](#);
- Запустите **.exe** файл для установки;
- После окончания процесса установки потребуется зарегистрироваться. Для простоты можно выбрать вариант регистрации с помощью существующего **google** аккаунта.

Для установки под **Mac** или **Linux** воспользуйтесь этими инструкциями:

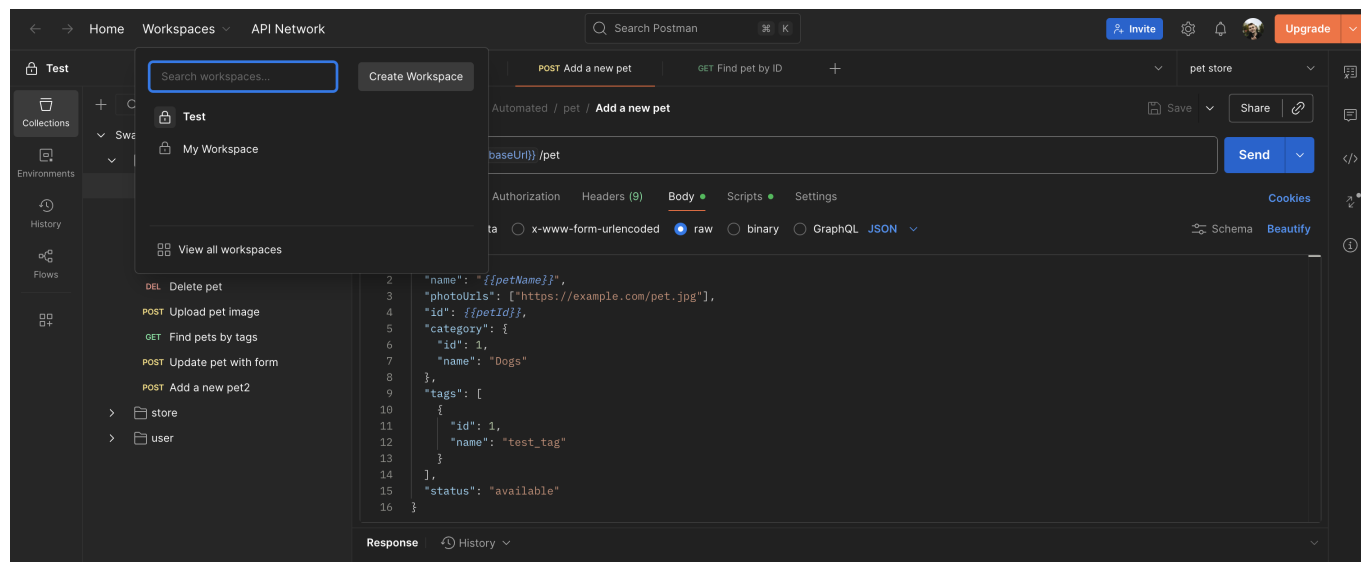
- [Установка для Mac](#);
- [Установка для Linux](#).

Также у **Postman** есть [web версия](#), которая работает прямо в браузере. Она доступна по этой [ссылке](#). На данный момент эта версия находится в процессе доработок, поэтому часть

возможностей настольной версии там недоступна.

Попробуйте установить себе подходящую версию **Postman**, чтобы в дальнейшем была возможность практиковать полученные знания.

Сейчас мы кратко рассмотрим основные элементы **Postman**, необходимые для начала работы. Но для этого нам потребуется создать новое рабочее пространство (**Workspace**). В верхнем меню выбираем соответствующий пункт и нажимаем кнопку **Create Workspace**.



В открывшемся окне вводим название нашего рабочего пространства. Пусть это будет **Postman for beginners**. В поле **Summary** можем добавить краткое описание. Затем можно указать уровень доступа. Т.к. я не планирую совместное использование своих запросов с кем-то еще, выбираю настройку **Personal**. Это значит, что созданный **workspace** будет доступен только мне. В конце нажимаем кнопку **Create Workspace**.

Create your workspace

Workspace name

Postam for beginners

Select workspace type



Internal



Build and test APIs within your team.



Partner

[Upgrade plan](#)

Share APIs securely with trusted partners.



Public



Make APIs accessible to the world.

Choose how you'd like to set up this workspace



Blank workspace



I want to start from scratch.



From a Git repository



I want to connect my existing Git project.



Choose a Folder

🚀 Create with Postman AI

Ask AI to set up collections, specs, and environments for your API.

Describe what you want to build (e.g., "Payments API").

Since you don't have a team, we will create one for you along with this workspace.

Everyone in your team can access ▼

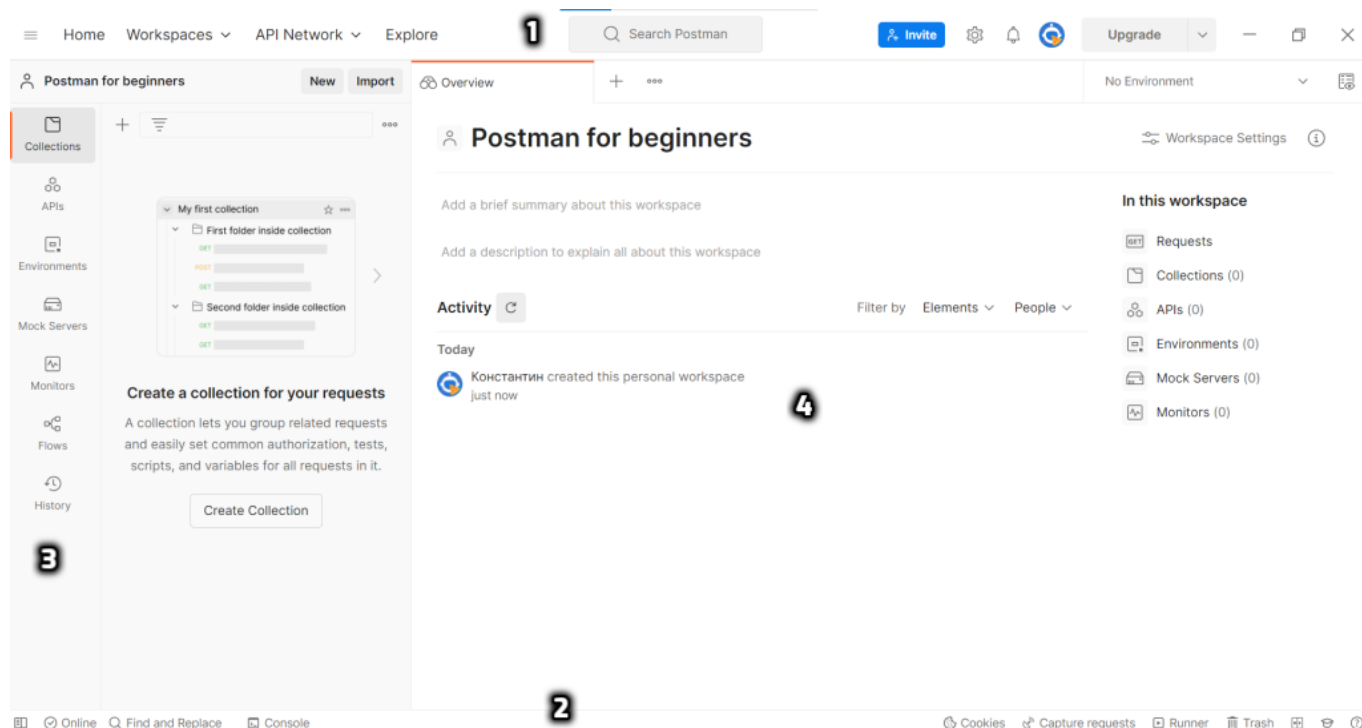
Cancel

Create Workspace

Теперь рабочее пространство создано и можем двигаться дальше. Перед нами окно приложения. Оно состоит из:

- верхнего меню;
- нижнего меню;
- бокового меню;
- основной зоны работы.

Верхнее меню дает возможность создания рабочего пространства (**Workspaces**) и доступа к различным **api (API Network)**. Также с его помощью можно делать поиск. **Home** - содержит вашу домашнюю страницу. На ней можно посмотреть вашу недавнюю активность. С помощью **Workspaces** - можно создать новое рабочее пространство, выбрать одно из недавно посещенных, либо сделать поиск по существующим.



В боковом меню доступны основные функциональные элементы Postman. В истории (**History**) вы можете увидеть сделанные ранее запросы. Для более удобного хранения, группировки и поиска таких запросов в Postman есть коллекции (**Collections**). К остальным элементам бокового меню мы вернемся позже. Если у вас небольшой экран, для удобной работы можно свернуть боковое меню с помощью кнопки в нижнем левом углу.

Большая часть вашей работы в **Postman** будет проходить в основной рабочей зоне. Основные её части - вкладки, выпадающий список для выбора окружения и правое боковое меню. К двум последним мы вернемся позже. Вкладки нам нужны, для лучшей организации работы с запросами. Здесь **Postman** сделан по тому же принципу, что и браузеры, где новую страницу удобнее открывать в новой вкладке.

Далее мы рассмотрим другие элементы основной рабочей зоны и отправим первый запрос с помощью **Postman**.

Основная функциональность Postman — возможность создания и отправки запросов к API для проверки его функциональности и получения данных. Для этого не потребуется писать код или команды в терминале. В интерфейсе Postman вы создаете запрос, нажимаете кнопку «Отправить» и получаете ответ от **API**. **API расширяется как Application Programming Interface** или программный интерфейс приложения. С его помощью мы можем получить доступ к возможностям другого приложения и обмениваться с ним данными. Такое приложение называется **API-сервером**. Отправка запросов и получение ответов происходит через интернет с помощью протокола **HTTP**. Приложение, которое отправляет запрос, называется клиентом. Это может быть мобильное приложение, web-сайт или другой сервис.

Запрос всегда содержит **URL** вызываемого эндпоинта **API** и **HTTP** метод запроса. Эндпоинт — это **URL**, который предоставляет доступ к определенной функциональности сервиса. Он позволяет клиентским приложениям отправлять запросы на сервер и получить доступ к данным или выполнить определенные операции. Каждый эндпоинт обычно связан с определенным **HTTP-методом**, таким

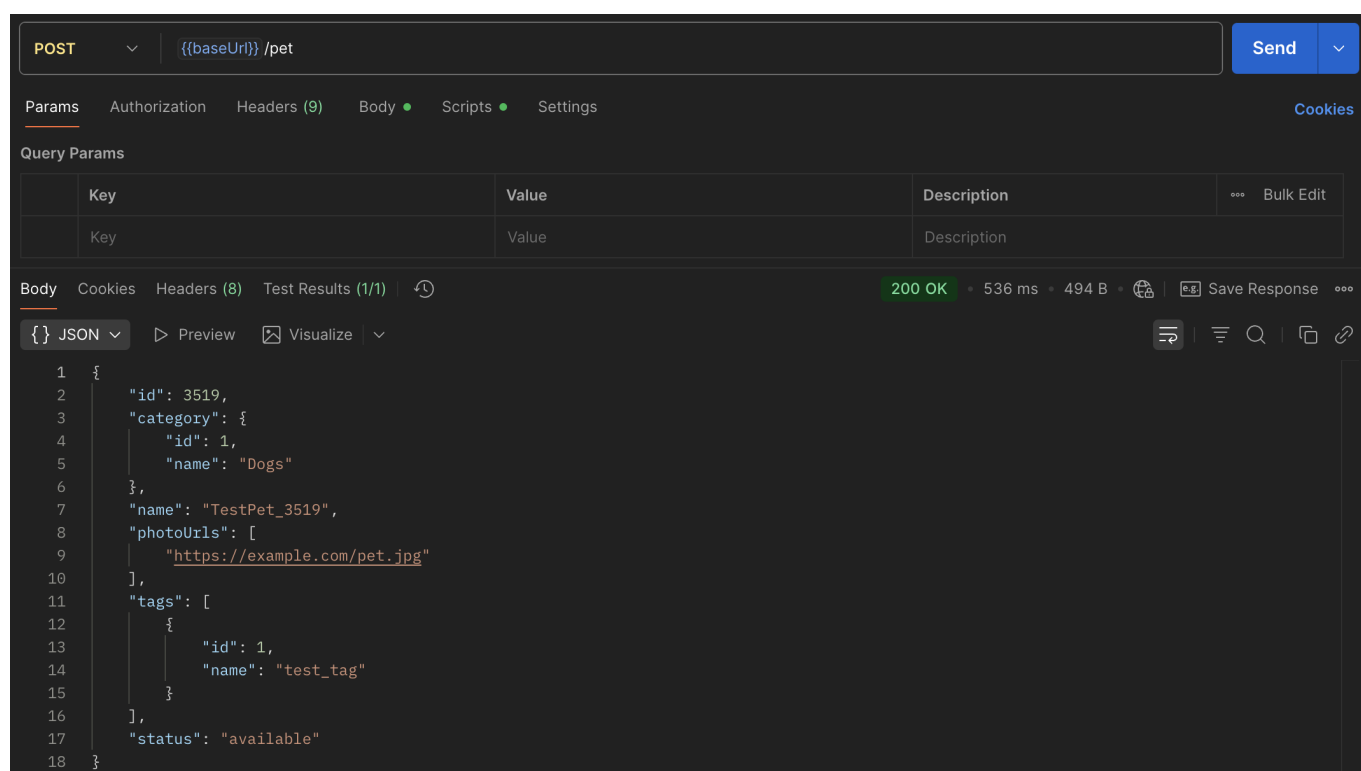
как, например, **GET** или **POST**, который определяет тип операции, которую можно выполнить с помощью этого эндпоинта. Наиболее часто используются следующие методы:

- **POST** - для добавления новых данных
- **GET** - для чтения данных
- **PUT** - для обновления данных
- **PATCH** - для частичного обновления данных
- **DELETE** - для удаления данных

Давайте попробуем отправить первый запрос. Это будет **POST** запрос из демонстрационного **API Swagger Petstore**. Подробнее с ним вы сможете познакомиться по [ссылке](#). Выглядит запрос следующим образом:

```
https://petstore.swagger.io/pet
```

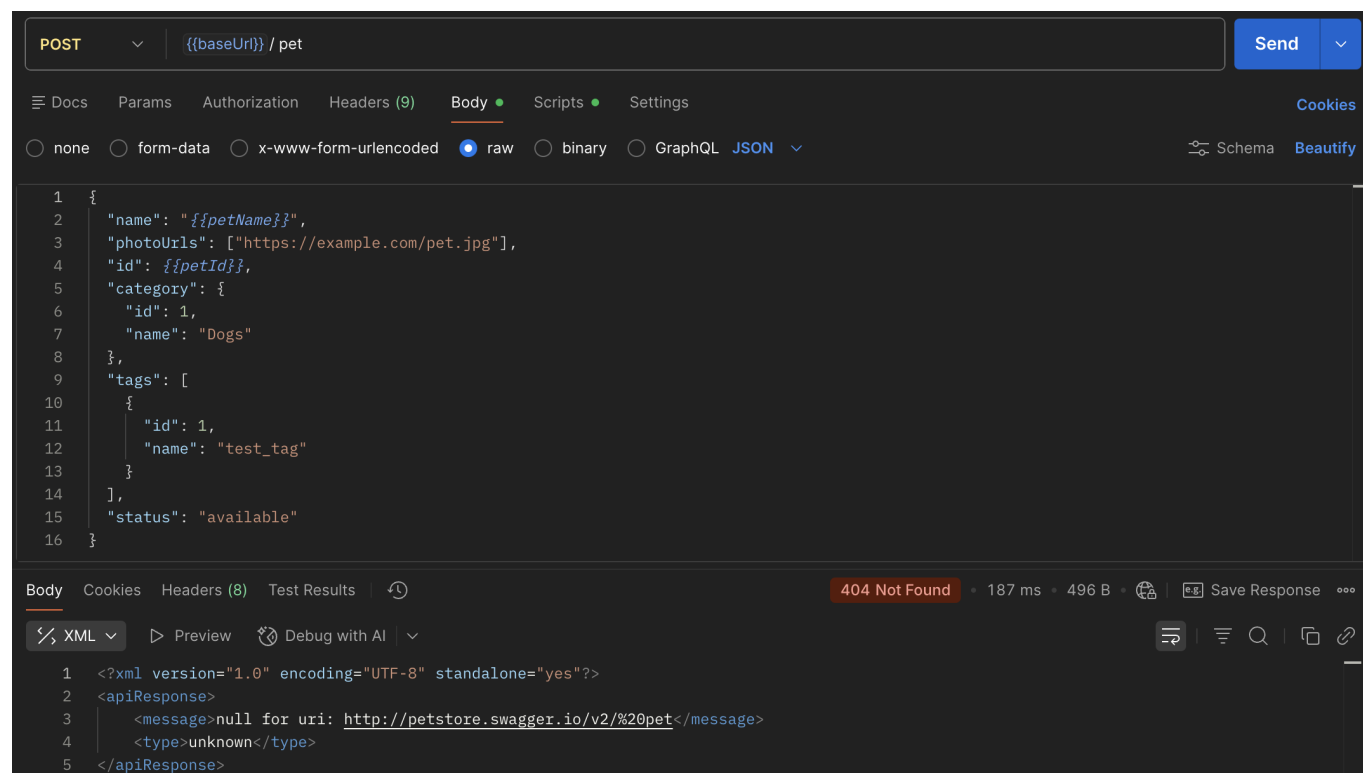
Для отправки в Postman выбираем метод **POST**. Копируем адрес нашего эндпоинта и нажимаем кнопку **Send**. В результате получаем ответ, содержащий данные в виде **json** объекта.



Здесь мы можем увидеть тело ответа (**Body**), который мы получили от сервера. Postman автоматически распознает формат **JSON** и форматирует его для лучшего восприятия. Помимо **Body**, мы можем посмотреть куки (**Cookies**) и заголовки (**Headers**) ответа, нажав на соответствующие ссылки. Также мы видим, что наш запрос выполнен успешно и в ответе мы получили код ответа **200 OK**. На выполнение запроса ушло 700 мс. Содержимое в теле ответа, время обработки и размер могут отличаться от примера выше, т.к. на сервисе добавляются новые записи и удаляются старые.

Многие учащиеся копируют адрес запроса и вставляют в **Postman**. Если вы делаете так же, обратите внимание, чтобы адрес скопировался без посторонних символов. Часто вместе с адресом

копируются пробелы или переносы строк. В этом случае в результате выполнения запроса, вы получите ответ **404 Not Found**.



Задание:

1. Написать тесты в **Postman** по <https://petstore.swagger.io/>
2. Реализовать 3 основные папки (**Pet**, **Store**, **User**). См. Рисунок ниже.

pet Everything about your Pets		Find out more ^
POST	/pet/{petId}/uploadImage uploads an image	🔒 ▼
POST	/pet Add a new pet to the store	🔒 ▼
PUT	/pet Update an existing pet	🔒 ▼
GET	/pet/findByStatus Finds Pets by status	🔒 ▼
GET	/pet/findByTags Finds Pets by tags	🔒 ▼
GET	/pet/{petId} Find pet by ID	🔒 ▼
POST	/pet/{petId} Updates a pet in the store with form data	🔒 ▼
DELETE	/pet/{petId} Deletes a pet	🔒 ▼
store Access to Petstore orders		^
GET	/store/inventory Returns pet inventories by status	🔒 ▼
POST	/store/order Place an order for a pet	▼
GET	/store/order/{orderId} Find purchase order by ID	▼
DELETE	/store/order/{orderId} Delete purchase order by ID	▼
user Operations about user		Find out more about our store ^
POST	/user/createWithList Creates list of users with given input array	▼
GET	/user/{username} Get user by user name	▼
PUT	/user/{username} Updated user	▼
DELETE	/user/{username} Delete user	▼
GET	/user/login Logs user into the system	▼
GET	/user/logout Logs out current logged in user session	▼
POST	/user/createWithArray Creates list of users with given input array	▼
POST	/user Create user	▼

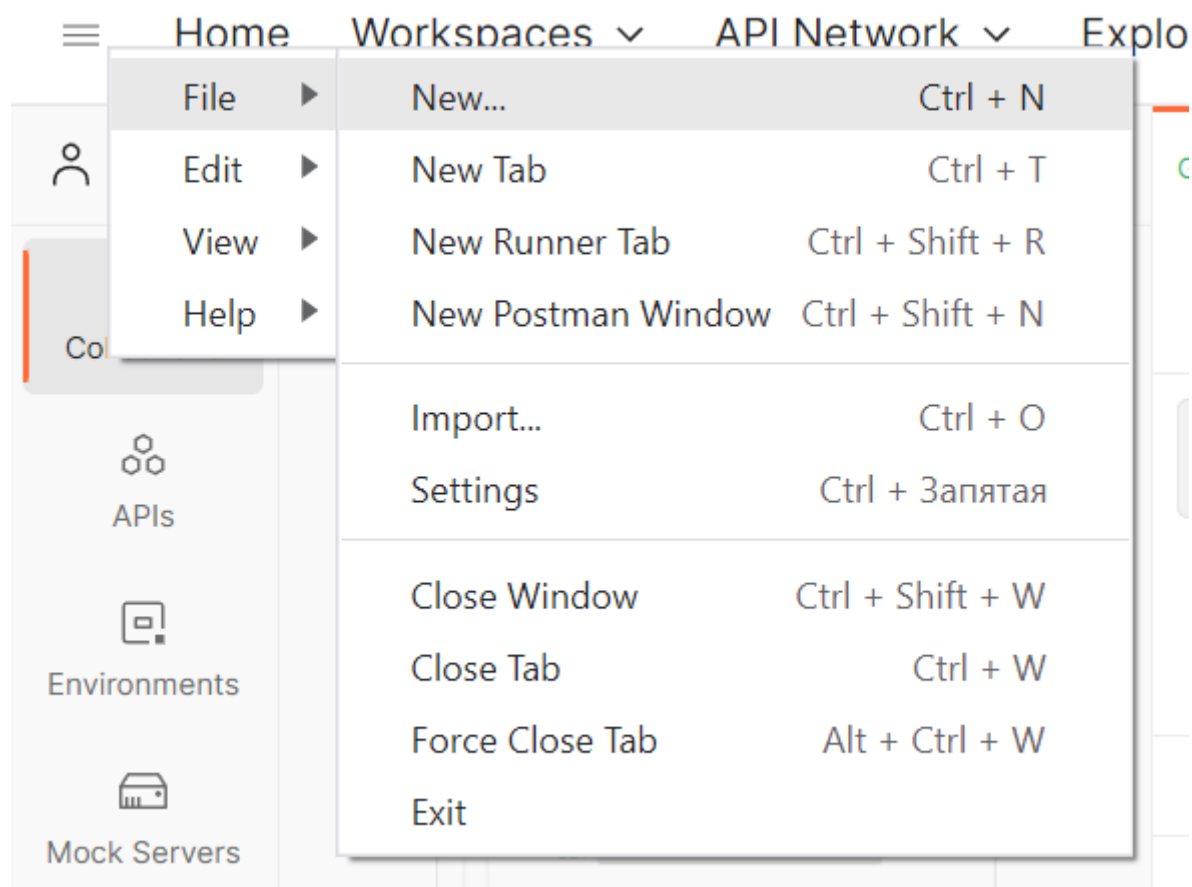
3. Прописать эндпоинты.

4. Сделать автотестирование, используя «**Run collection**».

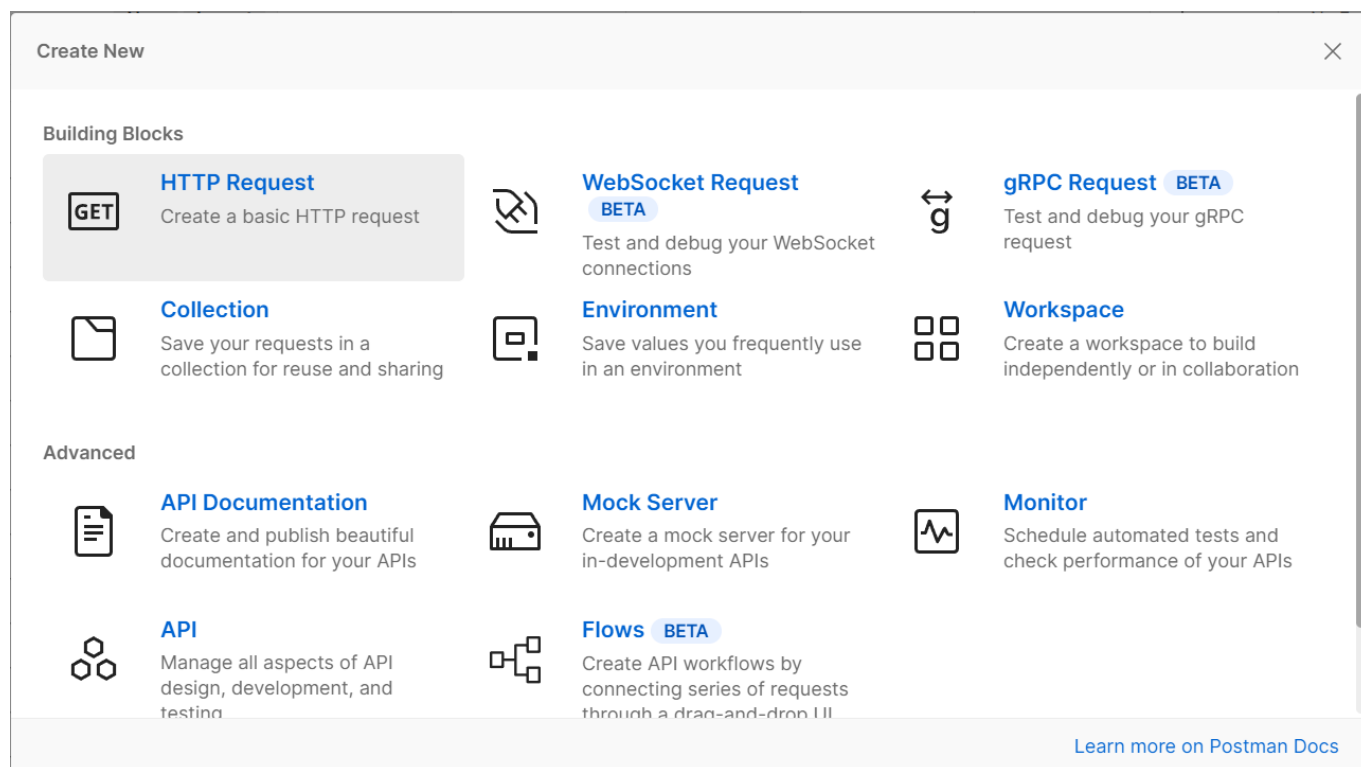
Работа с запросами и ответами в Postman

Запрос может содержать различную информацию, которая определяет, какие данные **Postman** отправит при взаимодействии с **API**. Обязательно нужно указать метод запроса и его **URL**. С их помощью определяется **API эндпоинт**, который предоставляет доступ к определенной функциональности сервиса, с которой **Postman** будет взаимодействовать.

Для начала давайте попробуем отправить основные типы **HTTP запросов**.



Создадим новый запрос. Для этого в верхнем меню нужно выбрать **File > New...**



В появившемся окне в разделе **Building Blocks** выбираем **HTTP Request**. Либо можно рядом с существующими вкладками запросов нажать **+** и создать новую вкладку.

GET запрос.

Попробуем отправить **GET** запрос. **GET** - это один из основных методов **HTTP** протокола. Обычно такой тип запросов используется для чтения данных. В **GET** запросе клиент запрашивает определенный ресурс, который находится на сервере, и получает ответ от сервера в виде содержимого ресурса.

```
https://petstore.swagger.io/v2/store/inventory
```

Метод **GET** у нас уже выбран по умолчанию. Осталось указать **URL** нашего **API**. После того, как мы начнем вводить его, Postman в выпадающем списке покажет нам ранее введенные адреса. В данном примере наш **URL** состоит из двух частей.

Первая из них **базовый (base) URL** - это **https://petstore.swagger.io**. Она состоит из протокола **https**. Здесь чаще всего будет использоваться либо **https**, либо **http** протокол. Затем идет **host**, представленный доменным именем **petstore.swagger.io**. Вместо него также может быть использован **ip адрес хоста**. И затем идет неявно указанный порт, в нашем случае это **443**.

Указанный неявно - означает, что если вы добавите к запросу порт:

https://petstore.swagger.io:443, ничего не изменится, т.к. этот же порт используется по умолчанию для протокола **https**. Если же ваш запрос использует протокол **http**, порт по умолчанию будет **80**. Если порт отличается от используемых по умолчанию, потребуется явно указать его через двоеточие.

Вторая часть это путь(**path**) запроса. Он находится сразу после **base URL**.

The screenshot shows the Postman interface with a GET request to `https://petstore.swagger.io/v2/store/inventory`. The interface includes a top bar with the request method and URL, a 'Send' button, and a table for query parameters.

KEY	VALUE	DESCRIPTION	...	Bulk Edit
Key	Value	Description		

После ввода **URL** мы можем отправить запрос. Для этого нужно нажать кнопку **Send**. Через некоторое время после этого в нижней части главного окна мы увидим ответ от сервера. Его содержимое мы рассмотрим позже, а пока давайте перейдем к другим запросам.

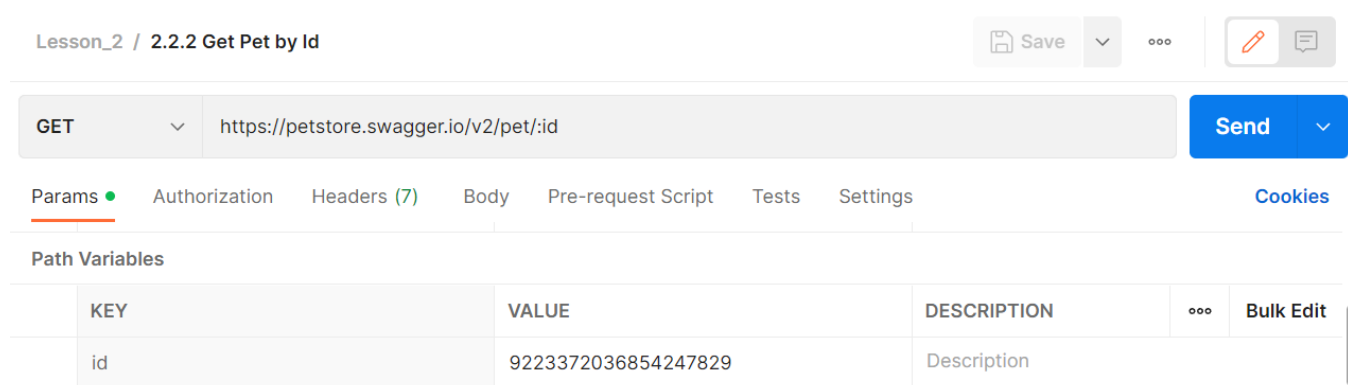
В предыдущем примере запрос не содержал параметров, но иногда их требуется передавать на сервер. Они могут понадобиться для передачи дополнительной информации в запросе и для уточнения его целей. Параметры могут использоваться для передачи данных, которые необходимы для выполнения операции, определенной методом **HTTP**, и могут варьироваться в зависимости от используемого метода. Например параметры могут использоваться для передачи настроек фильтрации, сортировки или поиска, чтобы получить только нужные данные. Так же они могут использоваться для передачи данных, которые необходимо добавить на сервер, для обновления или удаления существующих данных.

Таким образом, использование параметров в **HTTP** методах позволяет уточнить цели запроса, определить необходимые данные для выполнения операции и передать их на сервер. Это делает **HTTP** методы более гибкими и позволяет использовать их в различных сценариях, связанных с передачей данных через Интернет. В этом примере мы рассмотрим **path** параметры.

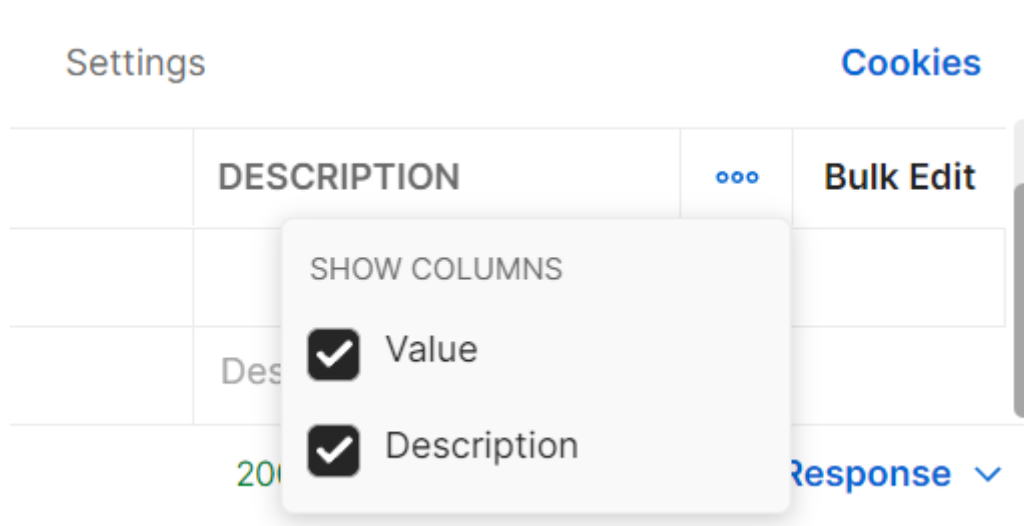
```
https://petstore.swagger.io/v2/pet/:id
```

Это параметры которые являются частью пути запроса. Они содержатся в части **URL** пути и используются для передачи данных, которые определяют определенный ресурс на сервере и его характеристики. **Path** параметр можно указать только в одном, строго указанном месте. В документации к **API** такие параметры часто задаются в фигурных скобках.

В данном примере мы будем использовать **API** для получения информации о домашнем питомце по его идентификатору в системе. Его мы будем передавать на сервер с помощью **path** параметра **id**. После отправки такого запроса, в ответе от сервера придёт информация о питомце, который имеет переданный нами идентификатор.

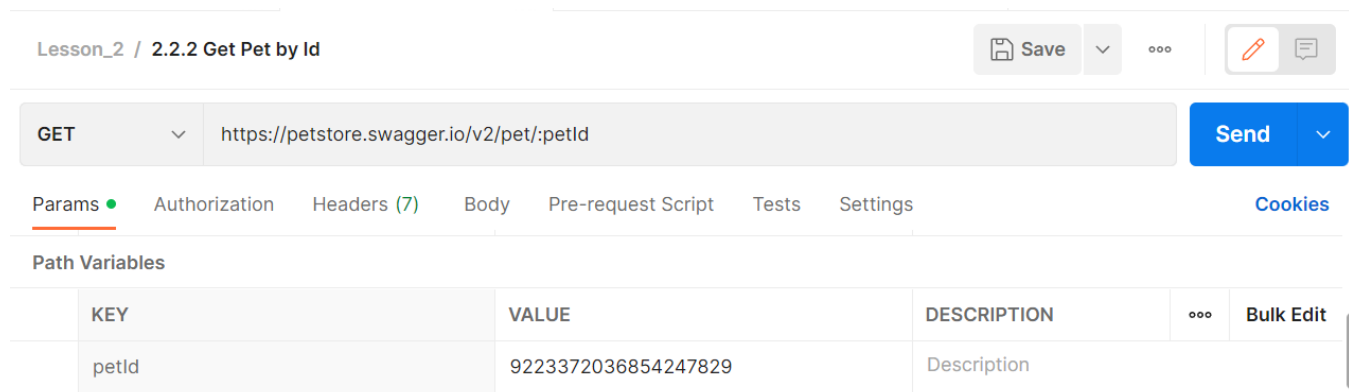


В **Postman** для добавления такого параметра в запрос, необходимо указать двоеточие перед его именем. После этого он появится на вкладке **Params**, в таблице **path** параметров. Там же затем можно будет задать его значение в столбце **value**. Оно будет использовано во время отправки запроса на сервер. Если **path** параметр присутствует в запросе, он является обязательным. При попытке оставить его значение пустым, в ответе от сервера вы увидите ошибку.

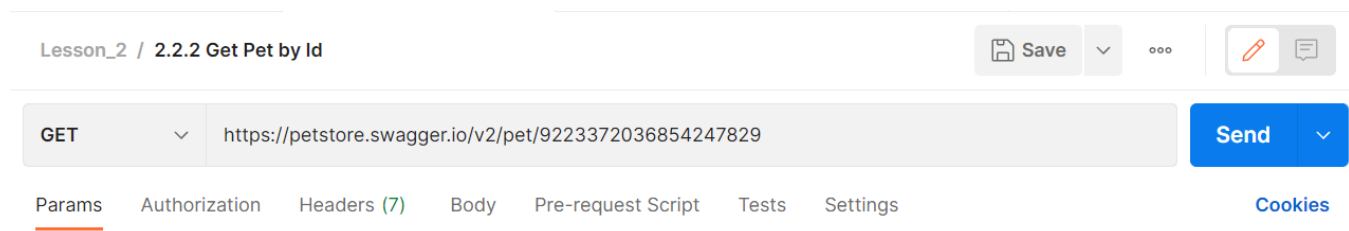


В таблице **Path Variables** кроме **value**, присутствует поле **description**. В нем вы можете добавить описание к каждому из параметров. Столбцы **value** и **description** можно убрать из таблицы. Для этого нужно нажать на три точки в правой части таблицы. В появившемся меню оставить галки только возле полей, которые нужно показывать в таблице параметров.

В прошлом примере мы использовали имя параметра **id**, для лучшего понимания. Можно использовать любое другое имя, результат выполнения запроса от этого не изменится.



Такой запрос будет аналогичен предыдущему и вернёт такой же ответ от сервера, не смотря на другое имя **path** параметра. Мы можем даже передать значение через **URL**, не используя дополнительных параметров.



Такой вызов будет таким же, как и 2 предшествующих варианта, хотя в нем мы не используем таблицу **path** параметров **Postman**. Вместо этого передаём значение параметра в **URL**.

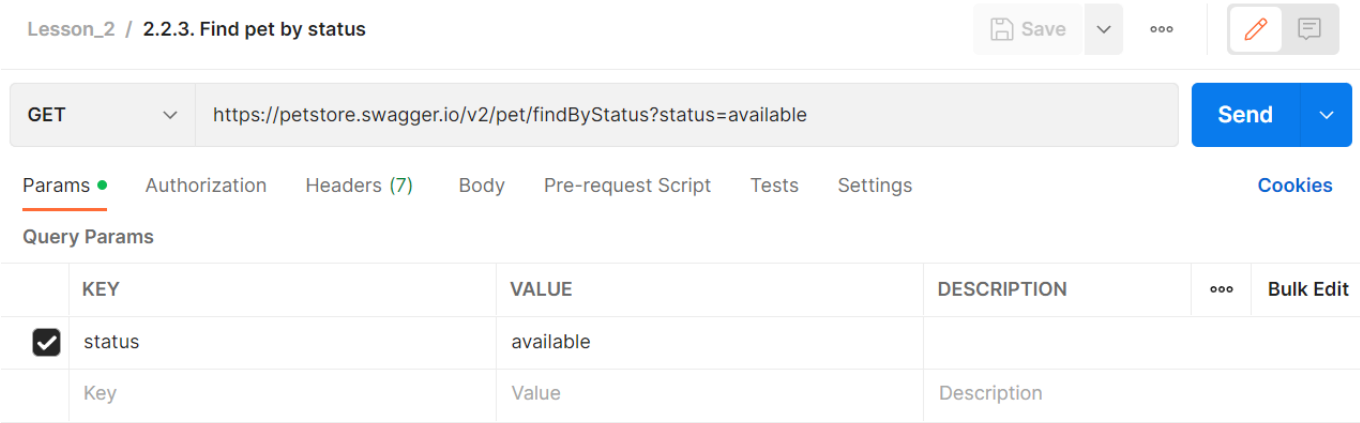
В прошлый раз мы рассмотрели отправку запроса с **path** параметрами. Теперь давайте остановимся на другом типе параметров, которые тоже передаются в строке запроса.

`https://petstore.swagger.io/v2/pet/findByStatus?status=available`

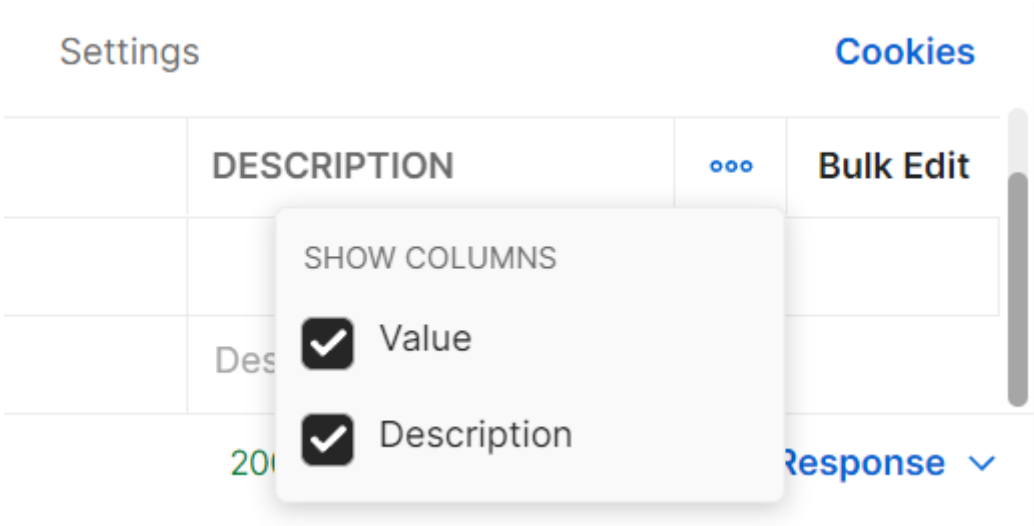
Они называются **query параметрами** или параметрами строки запроса. В **HTTP-запросах** они представляют собой дополнительные данные, которые передаются в **URL-адресе** и используются для передачи на сервер, значения для фильтрации, поиска, сортировки и т.д. **Query-параметры** могут использоваться вместе с любым **HTTP-методом**, но наиболее часто они используются с методом **GET**.

От основной части **URL** эти параметры отделяются знаком вопроса, а между собой знаком амперсанда. Каждый параметр представляет собой пару из имени и значения, разделенную знаком равенства. Имена и значения могут быть закодированы, используя механизм **URL-кодирования**, чтобы избежать ошибок при передаче специальных символов.

В данном примере мы используем запрос, который позволяет нам получить информацию о домашних питомцах отфильтрованную по одному из статусов. Значение статуса для фильтрации передается через **query параметр**. Его имя **status**, а значение **available**.



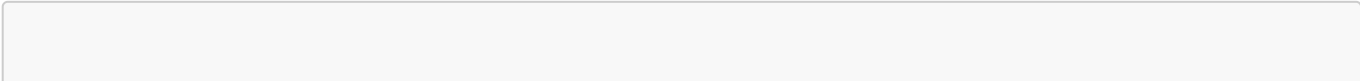
Если ввести такой параметр в строке запроса, он автоматически появится на вкладке **Params**, в таблице **Query Params**. Если добавить его на вкладке **Params**, тогда он появится в строке запроса. Еще на этой вкладке у каждого параметра есть поле **description**. В нем можно добавить описание к каждому из указанных параметров.



Как и для **path параметров**, здесь столбцы **value** и **description** можно скрыть. Для этого нужно нажать кнопку **View more actions** и выбрать только нужные поля.

Path и **query** параметры можно как совмещать внутри одного запроса, так и использовать по отдельности. В отличие от **path, query** параметры могут быть как обязательными так и необязательными. Обычно это указано в спецификации **API метода**. Например в нашем случае, **query** параметр **status** является обязательным. Поэтому выполнение запроса без него приведет к ошибке.

Еще одно отличие заключается в порядке указания параметров. Если ваш запрос содержит несколько **path параметров**, то каждый из них должен находиться в строго указанном месте. **Query параметры** могут быть указаны в произвольном порядке. На результат выполнения запроса это никак не повлияет.



```
https://petstore.swagger.io/v2/user/login?username=admin&password=123
```

```
https://petstore.swagger.io/v2/user/login?password=123&username=admin
```

Например результат выполнения указанных выше запросов будет одинаковым, хотя query параметры передаются в разном порядке.

Редактирование параметров через bulk edit.

В предыдущих примерах мы рассмотрели, как в **Postman** указать значения параметров в строке запроса или в таблицах **path** и **query** параметров. Есть еще один способ, который будет удобен в тех случаях, когда параметров много. Давайте рассмотрим запрос с несколькими параметрами.

```
https://petstore.swagger.io/v2/user/login?username=admin&password=123
```

Lesson_2 / 2.2.4 Get User Login Save ... Edit Comments

GET ▼ https://petstore.swagger.io/v2/user/login?username=admin&password=123 Send ▼

Params ● Authorization Headers (7) Body Pre-request Script Tests Settings Cookies

Query Params

	KEY	VALUE	DESCRIPTION	...	Bulk Edit
☒	username	admin			
☒	password	123			

Для его использования нужно нажать кнопку **Bulk Edit** в правой части таблицы **Query Params**. Вернуться обратно можно будет нажав на кнопку **Key-Value Edit**.

GET ▼ https://petstore.swagger.io/v2/user/login?username=admin&password=123 Send ▼

Params ● Authorization Headers (7) Body Pre-request Script Tests Settings Cookies

Query Params

Key-Value Edit

username:admin
password:123

В открывшемся окне можно вводить параметры. Каждый параметр вводится в новой строке. Имя и значение параметра разделяются двоеточием. Например ранее введенный нами параметры **username** и **admin** будут выглядеть таким образом:

```
username:admin  
password:123
```

Как и с предыдущим запросом, мы можем нажать кнопку **Send** для отправки и через несколько секунд внизу увидим ответ от сервера.

Иногда в запросе может потребоваться отправить больший объем данных, чем пара ключ и значение, как это было в прошлых примерах. Например, вы хотите добавить нового пользователя. Будет не очень удобно передавать информацию о нем в виде **path** или **query** параметров. Для таких случаев используется передача параметров в теле запроса. Обычно такой способ применяется с **POST**, **PUT** и **PATCH** запросами. Информация о пользователе будет передана в теле запроса в виде **XML** или **JSON**.

XML (Extensible Markup Language) — это язык разметки, который используется для представления и передачи данных. Он был разработан для того, чтобы облегчить обмен данными между различными системами. **XML** представляет собой текстовый формат, который содержит теги, атрибуты и значения. Теги используются для определения элементов данных, а атрибуты позволяют указывать дополнительные характеристики для этих элементов. Значения могут быть текстовыми или другими элементами данных.

```
<person>  
  <name>Artur</name>  
  <lastname>Dvoretsky</lastname>  
  <age>26</age>  
  <address>  
    <street>Lenina str.</street>  
    <city>Moscow</city>  
  </address>  
</person>
```

Этот объект в формате XML представляет человека с именем Artur, фамилией Dvoretsky, возрастом 26 лет, живущего в Москве, на улице Ленина.

JSON (JavaScript Object Notation) — это легковесный формат для передачи данных, основанный на синтаксисе объектов **JavaScript**. Он используется для обмена данными между клиентом и сервером в интернет-приложениях. **JSON** представляет данные в виде пар имени и значения, где имя — это строка, а значение может быть строкой, числом, логическим значением, массивом, объектом или **null**.

```
{  
  "name": "Artur",  
  "lastname": "Dvoretsky",  
  "age": 26,  
  "address": {  
    "street": "Lenina str.",
```

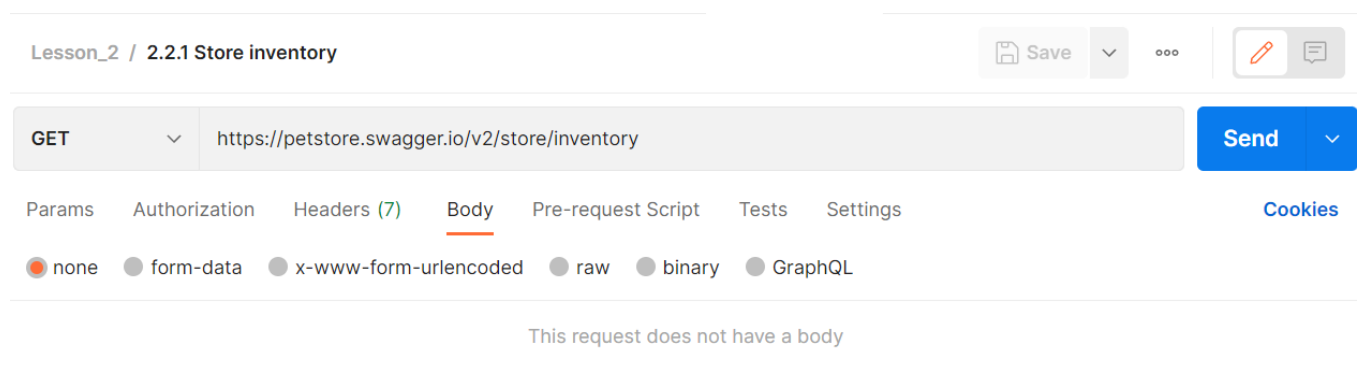
```
    "city": "Moscow"
  }
}
```

Этот объект в формате **JSON** аналогичен предыдущему. Также представляет человека с именем Artur, фамилией Dvoretzky, возрастом 26 лет, живущего в Москве, на улице Ленина.

Давайте рассмотрим основные отличия **JSON** и **XML**:

- Обычно данные в формате **JSON** имеют меньший размер, чем **XML-данные**, благодаря своему более компактному синтаксису.
- **JSON** использует более простой и компактный синтаксис, чем **XML**.
- **XML-файлы** обычно более читабельны, чем **JSON-файлы**, потому что **XML** использует явные теги и атрибуты, которые часто легче понимать, чем названия полей в **JSON**.
- **XML** позволяет более глубокую вложенность, чем **JSON**, что может быть полезно для некоторых типов данных.

В целом, **JSON** и **XML** имеют свои преимущества и недостатки, и выбор того, какой формат использовать, зависит от конкретных потребностей приложения.



Давайте вернемся в **Postman** и откроем вкладку **body**. На ней вы можете задать нужные данные, которые будут отправлены вместе с запросом. Там же можно выбрать тип отправляемых данных. В предыдущих запросах по умолчанию **Postman** для тела запроса использовал тип **none**.

Тип данных в теле запроса определяется из заголовка. Заголовки **HTTP (HTTP headers)** — это часть **HTTP-запроса** и ответа, которая передается между клиентом и сервером. Они содержат метаданные, необходимые для правильной обработки запроса и ответа. С помощью заголовков **HTTP** можно указать, например, тип передаваемого содержимого. Заголовок **content-type** в **HTTP** необходим для указания типа содержимого (например, текст, изображение, аудио или видео), передаваемого в теле запроса или ответа. Это позволяет получателю запроса или ответа правильно интерпретировать содержимое и выбрать соответствующий способ его обработки. Если этот заголовок отсутствует или указан неверно, это может привести к неправильной обработке содержимого или даже к ошибкам при его отображении. Поэтому правильное использование заголовка **content-type** очень важно для правильной обработки запросов и ответов в протоколе **HTTP**.

Если вы планируете отправлять данные в теле запроса, необходимо убедиться, что добавлен правильный заголовок **content-type**. Он нужен для того, чтобы **API** мог корректно обработать полученный запрос. Для типов **form-data** и **x-www-form-urlencoded** **Postman** автоматически

добавит в заголовки нужный `content-type`. Если вы планируете использовать тип `raw`, **Postman** добавит заголовок на основе выбранного вами типа (например, `json` или `text`). Для типа `binary` **Postman** не будет добавлять никаких заголовков. Заданный вручную заголовок `content-type` переопределяет значение, указанное **Postman**.

- `form-data` — этот формат в **HTTP** используется для передачи данных, которые представляют собой набор пар «ключ-значение». Такой тип часто используется при отправке данных формы на сайтах. Каждая пара соответствует одному элементу формы, например, текстовому полю, флажку и т.д. Еще с помощью формата `form-data` можно передавать файлы.
- Формат `x-www-form-urlencoded` внешне очень похож на `form-data`. Здесь данные также передаются в формате `ключ-значение`. Основное отличие от `form-data`: здесь мы не можем передавать файлы и указывать тип данных для каждого из полей.
- `Binary` — это формат, который предназначен для отправки бинарных данных. Он используется для передачи бинарных данных, таких как изображения, аудио, видео и другие файлы.
- `raw` — предназначен для отправки текстовых данных в различных форматах, таких как: `Text`, `JavaScript`, `JSON`, `HTML` и `XML`.
- `GraphQL` — позволяет отправлять запросы на языке запросов `GraphQL`. В рамках этого курса этот тип запросов не будет рассматриваться, но если вам интересно, можете ознакомиться с ним самостоятельно на [официальной странице GraphQL](#).

Добавление новых данных с помощью POST запроса

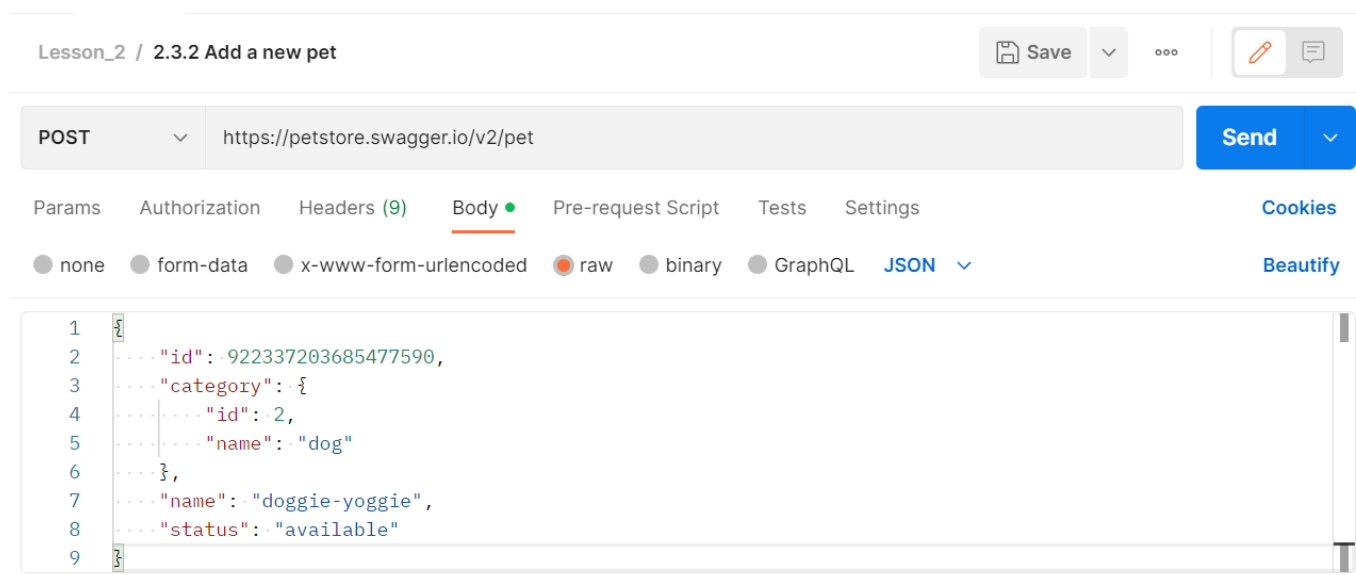
Давайте подробнее рассмотрим метод `POST`. Он является одним из методов запросов **HTTP**, используемых для отправки данных на сервер. Этот метод позволяет клиенту отправлять данные в теле запроса на сервер, который обрабатывает эти данные и возвращает ответ. Часто метод `POST` используется для отправки данных формы, таких как входные данные пользователя, поисковые запросы, комментарии и т. д. Кроме того, метод `POST` может использоваться для загрузки файлов на сервер.

Сейчас мы рассмотрим, как с помощью **Postman** отправить `POST` запрос для создания записи о новом домашнем животном в нашем тестовом API. Для этого будем использовать следующий эндпоинт:

```
https://petstore.swagger.io/v2/pet
```

```
{
  "id": 922337203685477590,
  "category": {
    "id": 2,
    "name": "dog"
  },
  "name": "doggie-yoggie",
  "status": "available"
}
```

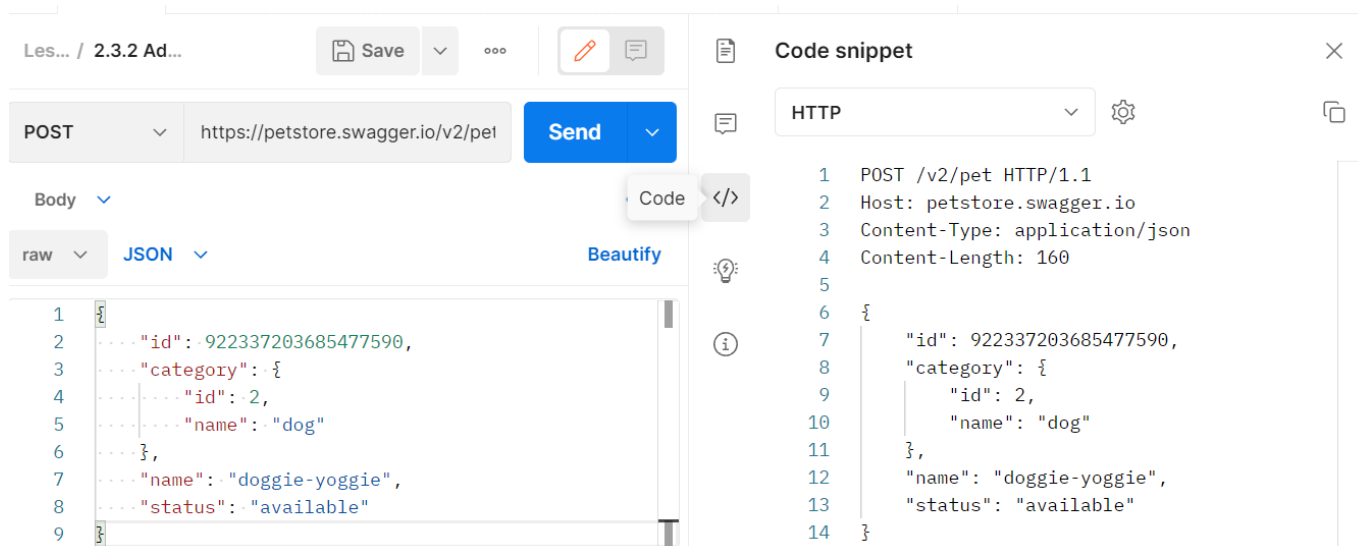

В отличие от предыдущих примеров, этот запрос не содержит параметров в строке запроса. Здесь мы будем передавать данные для создания записи в теле запроса. Запись о новом домашнем животном будет представлена в виде **JSON-объекта**.



На вкладке **Body** мы выбираем тип `raw`. Затем в выпадающем списке из всех значений выбираем **JSON**. Это нужно для того, чтобы **Postman** мог подставить нужный тип контента в заголовок запроса. Кроме того, это улучшит отображение объекта в окне ввода, т. к. **Postman** включит подсветку синтаксиса. Для данного запроса объект будет иметь следующий вид:

Поле `id` - это идентификатор домашнего животного. Здесь может быть указано любое число. Если вы захотите добавить свое домашнее животное поменяйте его значение. Только не вводите слишком длинных чисел, в этом случае запрос может выполняться с ошибкой. Далее идет категория, в которой задается её идентификатор и название. Затем в поле `name` мы задаём имя нашего животного. В последнем поле задается статус. Там можно указать одно из трех значений : `available`, `pending` или `sold`.

Кстати в **Postman** есть одна полезная фиша. Открыв в правом боковом меню пункт **Code**, вы можете посмотреть как будет выглядеть ваш **HTTP запрос**. Для этого в выпадающем списке нужно выбрать **HTTP**. Там также можно найти и другие формы запроса, например **CURL**, но сейчас мы не будем на них останавливаться подробно, а рассмотрим наш запрос. Именно в такой форме наш запрос отправляется на сервер для обработки.



В начале идет стартовая строка. В ней мы видим метод нашего запроса - **POST**. Затем идёт наш эндпоинт `/v2/pet`. **HTTP/1.1** - означает версию используемого протокола. В нашем случае это **HTTP 1.1**. Она определяет структуру следующих за стартовой строкой данных. Затем идут заголовки запроса, в которых передается адрес хоста `petstore.swagger.io`, на который мы отправляем запрос. Тип наших данных `application/json`, в которых мы передаём информацию о домашнем питомце, которого мы хотим добавить. И длина этих данных в заголовке **Content-Length**, которую **Postman** сам посчитал и добавил в заголовки запроса. Затем идет тело запроса, которое отделяется от заголовков пустой строкой. В нем передается **JSON**, который мы задали в окне **body**.

Обновление с помощью метода PUT

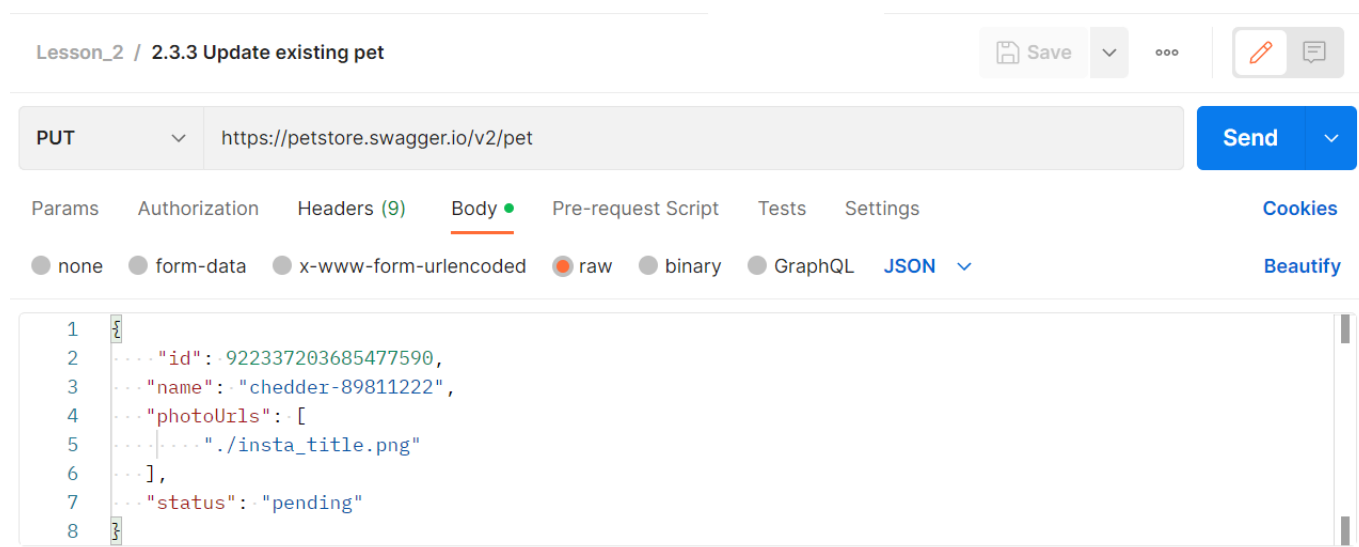
Мы добавляли нового питомца используя метод **POST** с данными в теле запроса. Теперь попробуем отправить **PUT** запрос. Обычно он используется для того, чтобы обновить существующие данные. Наш **PUT** запрос будет выглядеть следующим образом:

```
https://petstore.swagger.io/v2/pet
```

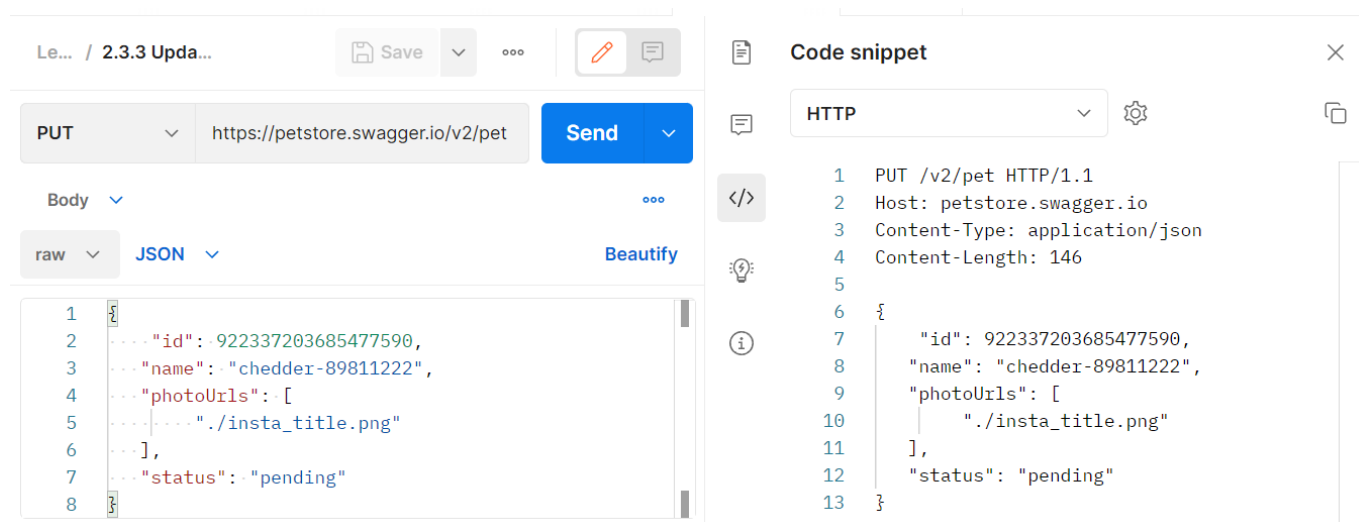
```
{
  "id": "922337203685477590",
  "name": "chedder-89811222",
  "photoUrls": [
    "./dog.png"
  ],
  "status": "pending"
}
```

В данном примере идентификатор в поле **id** совпадает с созданным. Имя питомца мы хотим изменить, поэтому в поле **name** указано новое значение. Также мы изменяем статус, поэтому здесь вместо `available` указано значение `pending`. Также мы добавляем параметр **photoUrls**, имеющий ссылку на картинку и убираем категорию, которую указывали при создании.

Как и в предыдущем примере, здесь в теле запроса можно передавать текстовые данные, различных форматов. Для этого используется вариант передачи данных: **raw**. В выпадающем меню можно выбрать формат передаваемых данных. Доступные форматы: **Text**, **JavaScript**, **JSON**, **HTML** и **XML**. В этот раз мы снова будем передавать **JSON**. Он будет содержать обновленные сведения о нашем домашнем питомце. Для данного запроса объект будет иметь следующий вид:



На вкладке **Body** мы вновь выбираем тип **raw**. Затем в выпадающем списке из всех значений выбираем **JSON** и **Postman** автоматически подставит нужный тип контента в заголовок запроса. Кстати в **Postman** для данных в формате **JSON** и **XML** появляется кнопка **Beautify**. С ее помощью можно переформатировать текстовые данные для красивого отображения в окне ввода.



Теперь давайте посмотрим, как наш запрос будет выглядеть во время отправки на сервер. Для этого выберем **Code** в правом боковом меню. Там видим, что в этот раз у нас другой метод запроса, т.к. обновление данных у нас происходит с помощью **PUT** и изменилось значение заголовка **Content-Length**, поскольку тело запроса стало короче. Ну и само тело запроса у нас изменилось. Отправим наш запрос на сервер, нажав на кнопку **Send**.

Lesson_2 / 2.2.2 Get Pet by Id

GET `https://petstore.swagger.io/v2/pet/:id` Send

Params • Authorization Headers (7) Body Pre-request Script Tests Settings Cookies

Path Variables

Key	Value
id	922337203685477590

Body Cookies Headers (8) Test Results 200 OK 172 ms 440 B Save as Example

Pretty Raw Preview Visualize JSON

```
1 {
2   "id": 922337203685477590,
3   "name": "chedder-89811222",
4   "photoUrls": [
5     "/insta_title.png"
6   ],
7   "tags": [],
8   "status": "pending"
9 }
```

Теперь давайте выполним **GET** запрос, одного из прошлых уроков. С его помощью получим данные нашего домашнего животного по его идентификатору. В ответе от сервера увидим, что теперь нам приходят обновленные данные. Подробнее ответы от сервера мы рассмотрим потом, здесь мы только хотели убедиться, что данные о питомце обновились.

Удаление данных с помощью DELETE запроса

В протоколе **HTTP DELETE** запрос используется для удаления указанного ресурса, находящегося на сервере. Это может быть, например, удаление учетной записи пользователя или удаление файла с сервера. Таким образом, **DELETE-запрос** может быть полезен во многих ситуациях, когда необходимо удалить ресурс с сервера. Далее мы рассмотрим, как с помощью **Postman** отправить **DELETE** запрос для удаления записи о существующем домашнем животном в нашем тестовом **API**. Для этого будем использовать следующий эндпоинт:

```
https://petstore.swagger.io/v2/pet/:id
```

В отличие от предыдущих примеров, этот запрос не содержит параметров в теле запроса. Единственное, что будет нужно передать на сервер — это идентификатор питомца. Это делается с помощью path-параметра **id**. В **Postman** данный запрос будет иметь следующий вид:

Lesson_2 / 2.3.5 Delete a pet

DELETE ▼ https://petstore.swagger.io/v2/pet/:id Send ▼

Params ● Authorization Headers (7) Body Pre-request Script Tests Settings Cookies

Path Variables

Key	Value	...	Bulk Edit
id	922337203685477590		

Поле **id** — это идентификатор домашнего животного. Его можно взять из предыдущих запросов. Перед отправкой запроса на сервер, давайте посмотрим, как он будет выглядеть. Для этого откроем вкладку **Code** в правом боковом меню и выберем тип **HTTP**.

Les... / 2.3.5 Del...

DELETE ▼ https://petstore.swagger.io/v2/pet/ Send ▼

Params ▼ ...

Path Variables

Key	Value	...	Bulk Edit
id	922337203685477590		

Code snippet

HTTP ▼ ⚙️ 📋

```
1 DELETE /v2/pet/922337203685477590 HTTP/1.1
2 Host: petstore.swagger.io
```

Там мы увидим, что в этот раз метод запроса у нас **DELETE**, т.к. с его помощью происходит удаление данных в нашем **API**. Из заголовков запроса остался только **host**. Также исчезло тело запроса, поскольку здесь мы его не передаем. Теперь давайте отправим наш запрос на сервер, нажав на кнопку **Send**.

Lesson_2 / 2.3.5 Delete a pet

DELETE ▼ https://petstore.swagger.io/v2/pet/:id Send ▼

Params ● Authorization Headers (7) Body Pre-request Script Tests Settings Cookies

Path Variables

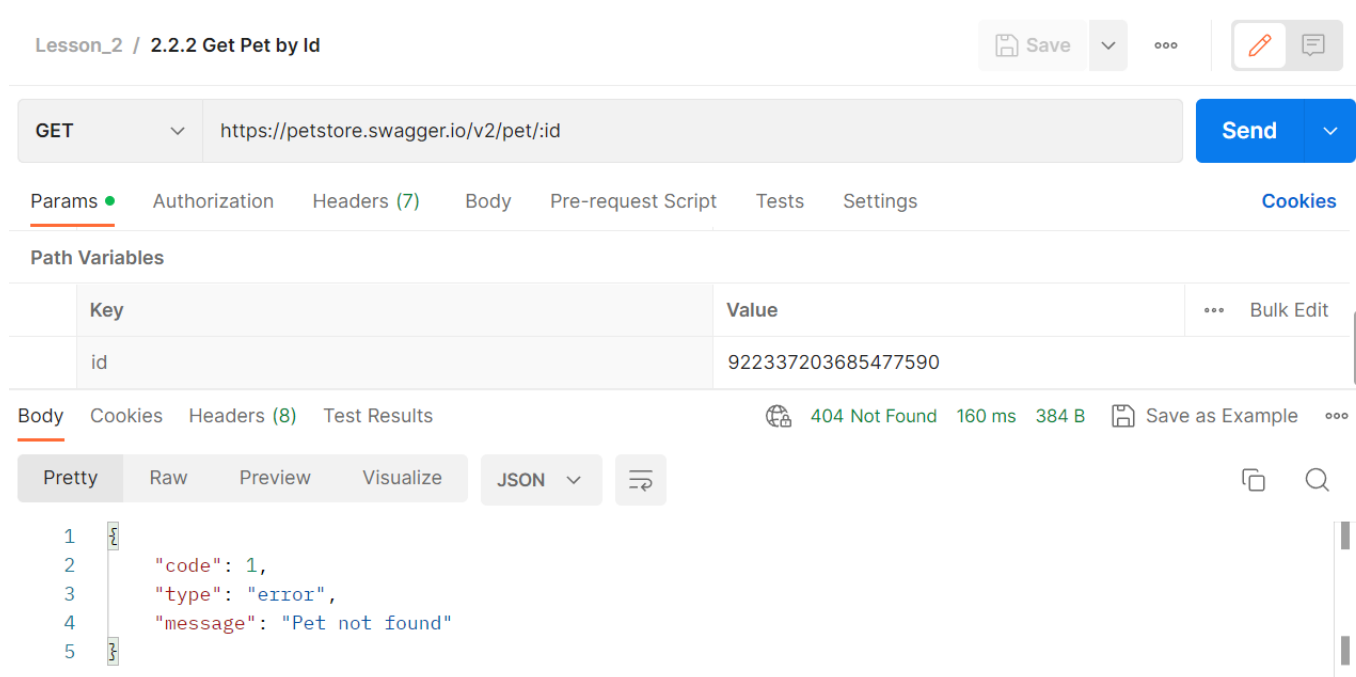
Key	Value	...	Bulk Edit
id	922337203685477590		

Body Cookies Headers (8) Test Results 🌐 200 OK 508 ms 386 B 💾 Save as Example ...

Pretty Raw Preview Visualize JSON ▼ 🔍

```
1 {
2   "code": 200,
3   "type": "unknown",
4   "message": "922337203685477590"
5 }
```

Теперь давайте выполним **GET** запрос, одного из прошлых уроков. С его помощью получим данные нашего домашнего животного по его идентификатору.



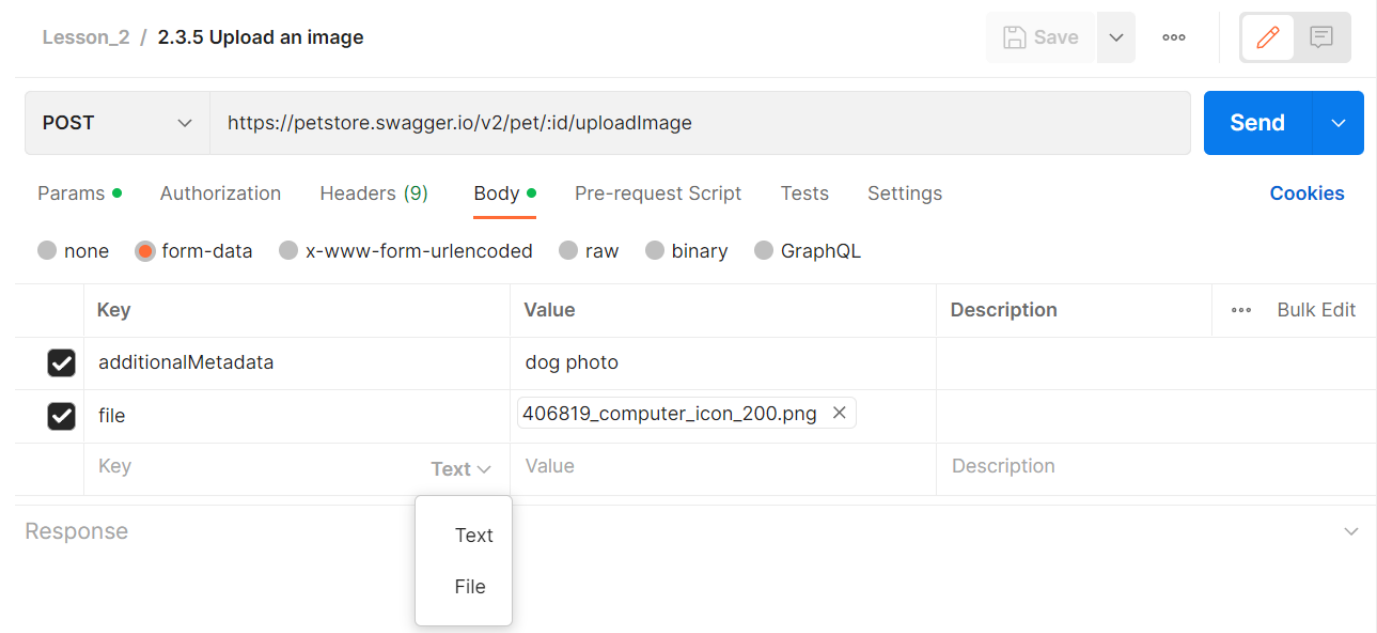
Идентификатор передаем тот же, что указали в запросе на удаление. В ответе от сервера увидим, что теперь нам приходит код ответа **404**. Он означает что домашний питомец с таким идентификатором не найден. В теле ответа мы так же получаем ошибку.

Передача данных в формате multipart/form-data

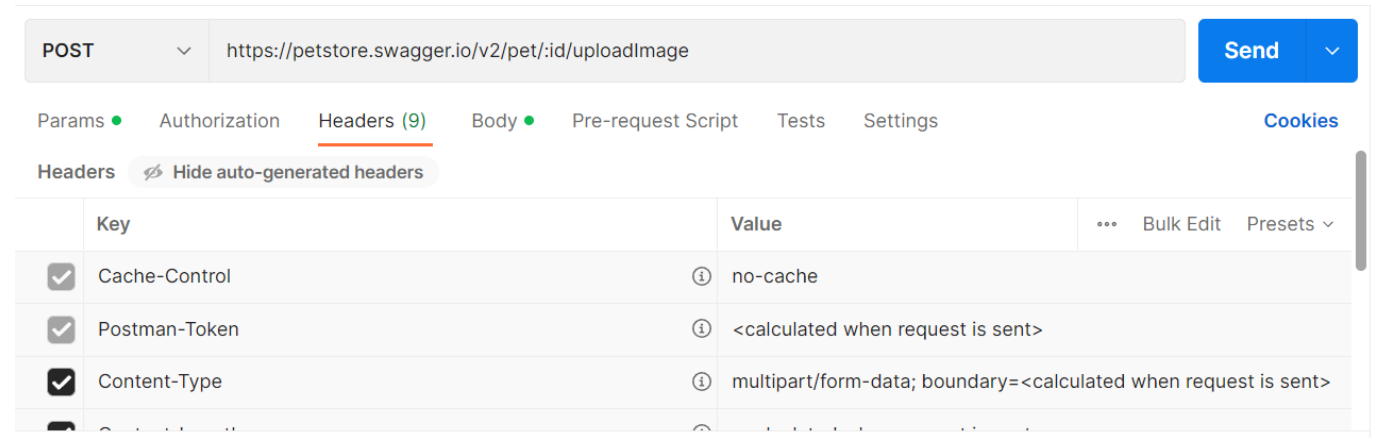
Мы рассмотрим еще один **POST** запрос. В нем мы снова будем обновлять данные нашего питомца, но в этот раз будет использован другой формат передачи данных. Он называется **multipart/form-data** и имеет такой же заголовок **content-type**. Такой тип контента в **HTTP запросах** может быть использован для отправки файлов или данных формы на сайтах. Когда вы отправляете такие данные, в браузере создается запрос, тело которого состоит из нескольких частей. Между ними размещаются границы (**boundary**), которые помогают серверу разделить данные на отдельные части. Давайте посмотрим, как отправить такой запрос, используя **Postman**.

```
https://petstore.swagger.io/v2/pet/:id/uploadImage
```

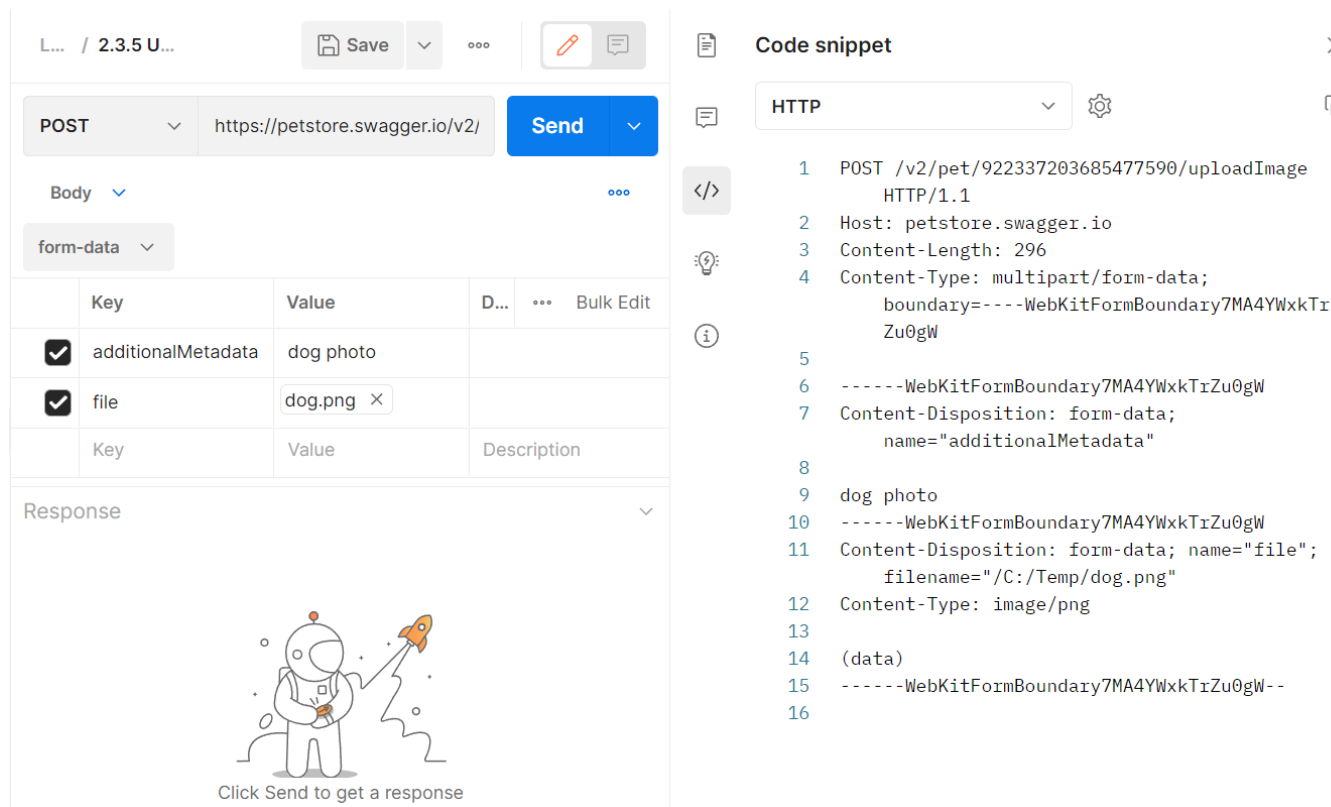
Будем использовать следующий эндпоинт. С его помощью мы можем добавить фото для нашего питомца. Идентификатор питомца, как и раньше, передается через **path-параметр**. Два других параметра — фото питомца и дополнительная информация, передаются через параметры в **form-data**.



В **Postman** для отправки подобного запроса на вкладке **body** необходимо выбрать тип **form-data**. Далее в таблице необходимо добавить данные, которые мы будем отправлять. Первый параметр называется **additionalMetadata**. В нем мы передаем текст, содержащий дополнительную информацию. Тип параметра оставим без изменений. Во втором параметре будем передавать фото питомца. Имя параметра — **file**. Затем в выпадающем меню нужно выбрать тип параметра — **file**. Теперь нужно нажать на **select files** и указать файл, который мы хотим передать на сервер. Настройка видимости полей в таблице и **bulk edit**. Здесь они работают аналогично **path** и **query** параметрам.



Откроем вкладку **headers**. Т.к. мы уже выбрали формат передачи данных **form-data**, **Postman** сам добавил в заголовки **content-type = multipart/form-data**. Рядом с ним динамически добавляется граница (**boundary**). **Postman** создаст её автоматически, чтобы потом подставить в тело запроса во время отправки на сервер.



The screenshot shows the Postman interface with a POST request to `https://petstore.swagger.io/v2/`. The request body is form-data with the following fields:

Key	Value	Description
additionalMetadata	dog photo	
file	dog.png	

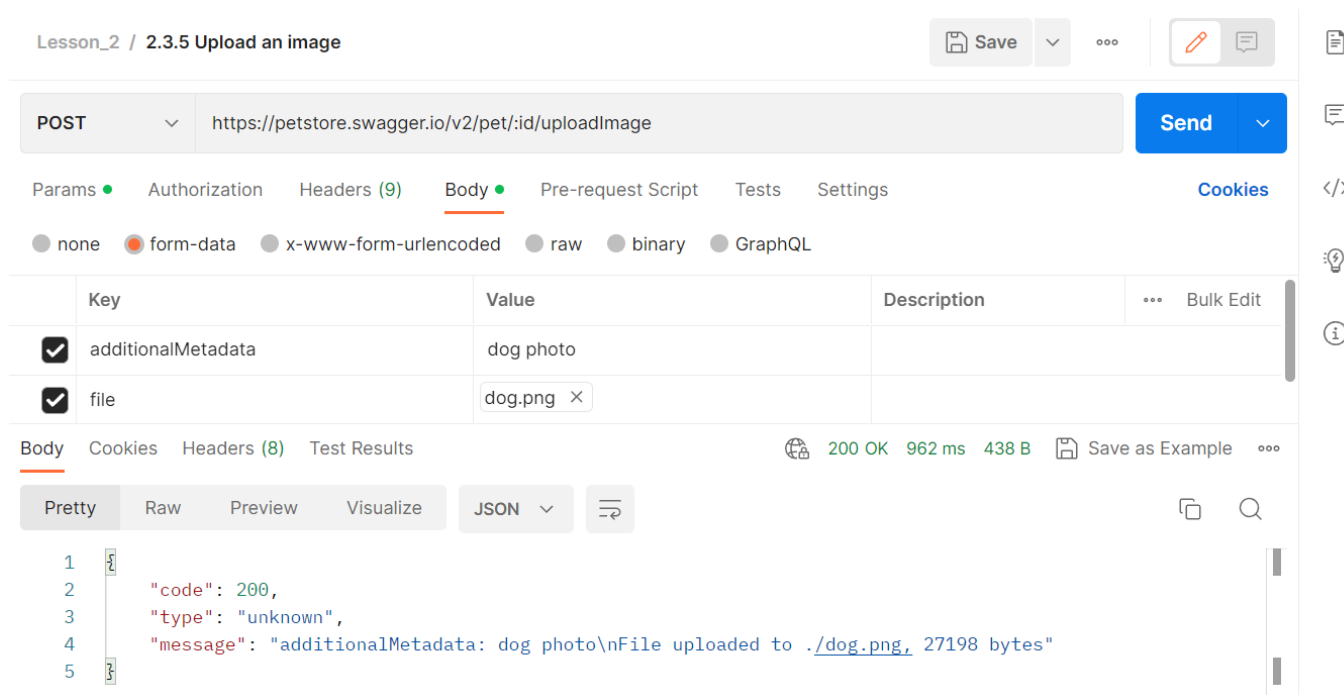
The response is a 200 OK status with a JSON body:

```

1 POST /v2/pet/922337203685477590/uploadImage
2 HTTP/1.1
3 Host: petstore.swagger.io
4 Content-Length: 296
5 Content-Type: multipart/form-data;
6   boundary=----WebKitFormBoundary7MA4YWxkTrZu0gW
7
8   dog photo
9   -----WebKitFormBoundary7MA4YWxkTrZu0gW
10  Content-Disposition: form-data; name="file";
11    filename="/C:/Temp/dog.png"
12  Content-Type: image/png
13  (data)
14  -----WebKitFormBoundary7MA4YWxkTrZu0gW--
15
16

```

Давайте посмотрим, как будет выглядеть наш запрос во время отправки на сервер. Для этого откроем вкладку **Code** в правом боковом меню и выберем тип **HTTP**. В первой строке видим метод **POST**, указанный в запросе эндпоинт и версию протокола **HTTP**. Затем в заголовках указаны host и длина контента. Следом в заголовке **content-type** указан наш тип контента **multipart/form-data** и **boundary**, которым в теле запроса будут разделяться его части. Затем идёт тело запроса. Каждая его часть отделяется от других строкой **boundary**, которая была указана выше.



The screenshot shows the Postman interface with the response of the POST request. The response is a 200 OK status with a JSON body:

```

1 {
2   "code": 200,
3   "type": "unknown",
4   "message": "additionalMetadata: dog photo\nFile uploaded to ./dog.png, 27198 bytes"
5 }

```

Отправим наш запрос, нажав на кнопку **Send**. В ответе видим успешный код **200 OK**. В теле ответа содержится **JSON** объект. Поле **message** содержит текст с данными, которые мы передали в запросе.

Передача данных в формате x-www-form-urlencoded

Данный формат используется для передачи данных в теле запроса, чаще всего для методов **POST** и **PUT**. Данные передаются в виде пар ключ-значение, разделенных символом "&", а ключ и значение разделяются символом "=". Значение каждого ключа и значения могут быть закодированы в формат **URL-кодирования**, чуть позже в примере мы рассмотрим, как это будет выглядеть. Этот формат является стандартным способом передачи данных в **HTML-формах**, а также используется в других **API** и веб-сервисах. Этот формат является простым и удобным для использования, поэтому он широко применяется в **HTTP-запросах**.

Этот формат похож на рассмотренный ранее **form-data**, но между ними есть следующие отличия:

- Все отправленные значения могут быть закодированы в формат URL-кодирования.
- В отличие от **form-data** здесь у нас нет возможности передавать файлы.
- Нельзя указать отдельный **content-type** для каждого из параметров.

```
https://petstore.swagger.io/v2/pet/:id
```

С помощью эндпоинта, который мы сейчас используем можно обновить некоторые данные нашего питомца. Идентификатор питомца снова передается через **path параметр**. Два других параметра - имя питомца и статус, передаются через параметры **x-www-form-urlencoded**.

Lesson_2 / 2.3.6 Update pet in the store

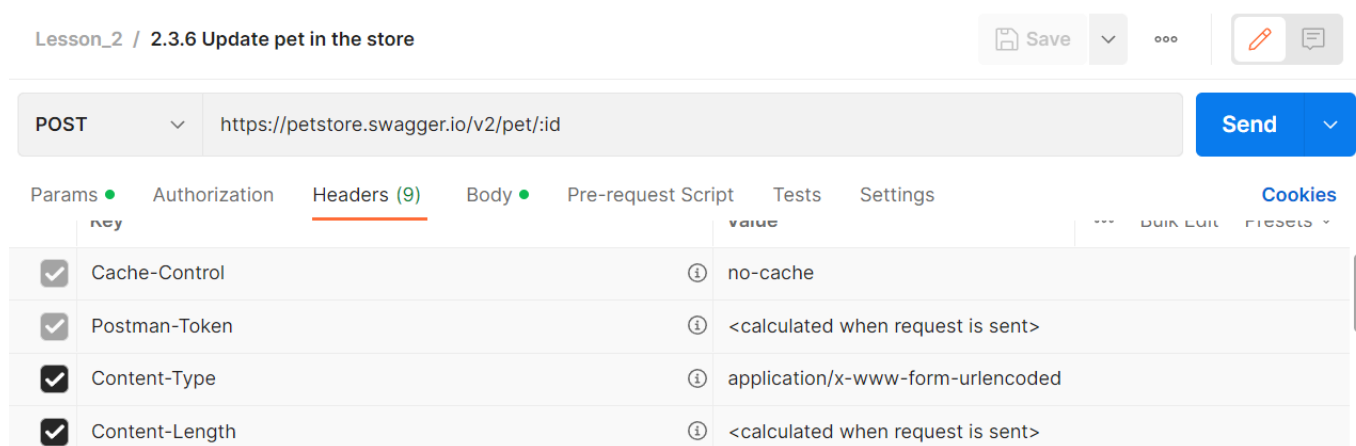
POST ▼ https://petstore.swagger.io/v2/pet/:id Send ▼

Params ● Authorization Headers (9) Body ● Pre-request Script Tests Settings Cookies

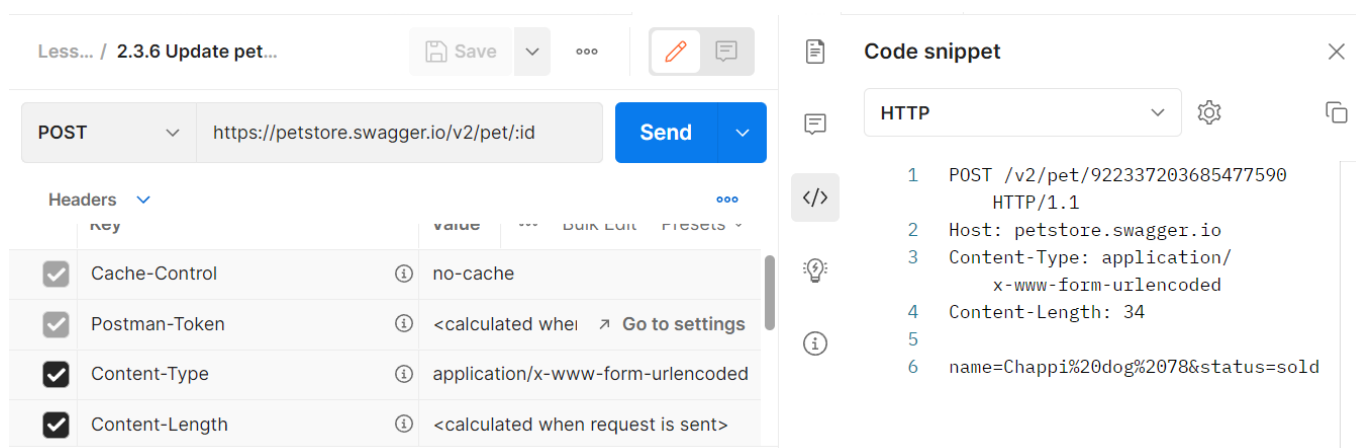
● none ● form-data ● x-www-form-urlencoded ● raw ● binary ● GraphQL

	Key	Value	Description	...	Bulk Edit
☒	name	Chappi dog 78			
☒	status	sold			

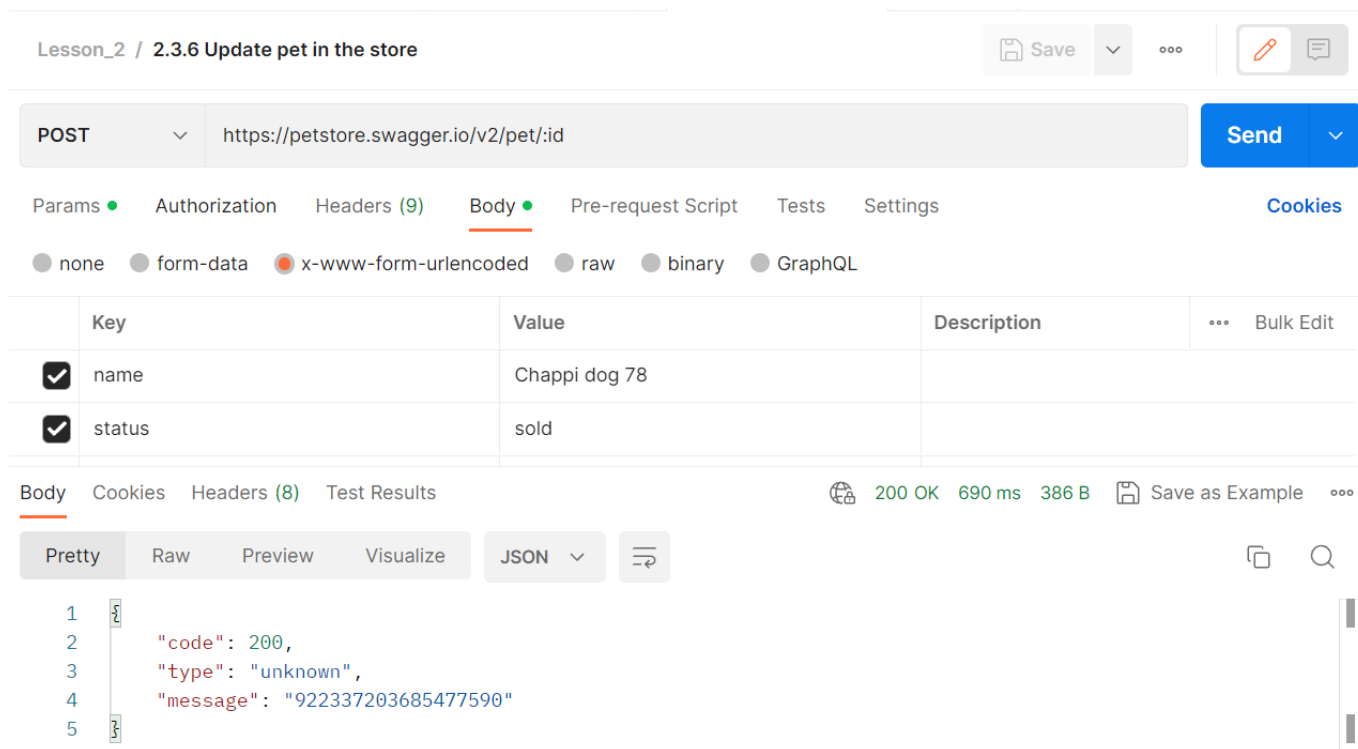
Для отправки подобного запроса в **Postman**, на вкладке **body** необходимо выбрать соответствующий тип **x-www-form-urlencoded**. Далее в таблице нам нужно указать данные, которые мы хотим отправить. Имя первого параметра name. В нем мы передаем новое имя нашего питомца. Во втором параметре будем передавать его статус. Имя параметра - **status**. Настройку видимости полей в таблице и **bulk edit** мы уже разбирали в прошлых уроках. Здесь они работают аналогично.



Перейдем на вкладку **headers**. Т.к. мы уже выбрали формат передачи данных **x-www-form-urlencoded**, **Postman** сам добавил в заголовки **content-type = application/x-www-form-urlencoded**. В отличие от типа **form-data**, здесь отсутствует граница (**boundary**), т.к. в теле запроса она не требуется.



Теперь посмотрим, как выглядит наш запрос при отправке на сервер. Откроем вкладку **Code** в правом боковом меню и выберем тип **HTTP**. В первой строке видим метод **POST**, указанный в запросе эндпоинт и версию протокола **HTTP**. Затем в заголовках указаны **host** и длина контента. В заголовке **content-type** указан наш тип контента **application/x-www-form-urlencoded**. Затем идёт тело запроса. Теперь оно выглядит иначе, все параметры идут в виде пар ключ-значение. Между собой они разделены символом "&", а ключ и значение разделяются символом "=". Обратите внимание на значение первого параметра **name**. В нем вместо пробела идёт значение **'%20'**. Это и есть **URL-кодирование**, о котором мы говорили ранее.



Отправим наш запрос, нажав на кнопку **Send**. В ответе видим успешный код **200 OK**. В теле ответа содержится **JSON объект**. Поле `message` содержит идентификатор домашнего питомца, который мы передали в запросе.

Работа с ответами

До сих пор мы рассматривали запросы к **API**. Теперь давайте подробнее остановимся на том, что нам приходит в ответе от сервера. **Postman** поможет нам с просмотром ответов. В нем есть возможность визуализации тела ответа, а также проверки его корректности. Основные части ответа — это код ответа, заголовки и тело ответа. Рассмотрим их более детально.

`https://postman-echo.com/post?name=anna`

Воспользуемся одним из предыдущих запросов. Укажем тип и **url** запроса, нажмем кнопку **Send**. Через некоторое время после этого в нижней части главного окна мы увидим ответ от сервера. Его содержимое мы будем рассматривать далее.

В **Postman** тело ответа мы можем посмотреть в нижней части главного экрана, на вкладке **body**. Она предоставляет несколько инструментов, которые помогут эффективнее работать с ответами от **API**. Это 4 вида представления тела ответа: **Pretty**, **Raw**, **Preview** и **Visualize**. В первых двух доступна функция поиска в запросе. Для открытия окна поиска нужно будет выбрать иконку поиска или нажать сочетание клавиш **⌘+F** или **Ctrl+F**. В **Postman** доступен автоматический показ изображений в теле ответа, если он содержит соответствующий заголовок **content-type**.

Форматированный (**Pretty**) вид — представление тела запроса для форматов **JSON** и **XML**. Этот вид автоматически форматирует полученный ответ для более легкого восприятия. Применяется удобная подсветка различных частей текста.

```

1 {
2   "args": {
3     "name": "anna"
4   },
5   "data": {"key": 10, "value": "book"},
6   "files": {},
7   "form": {},
8   "headers": {
9     "x-forwarded-proto": "https",
10    "x-forwarded-port": "443",
11    "host": "postman-echo.com",
12    "x-amzn-trace-id": "Root=1-62b44b88-11e3a41e211253ac16941db9",
13    "content-length": "43",

```

Postman делает форматирование автоматически, на основе пришедшего заголовка **content-type**. Для принудительного изменения форматирования можно воспользоваться выпадающим списком для выбора иного формата, рядом с кнопкой **Visualize**.

Если вы работаете только с форматом **JSON**, можно выставить принудительное форматирование в этом формате. Для этого в верхнем меню нужно выбрать раздел настроек (**Settings**). На вкладке **General** найти настройку определения языка (**Language detection**). Затем поменять ее значение с **Auto** на **JSON**.

```

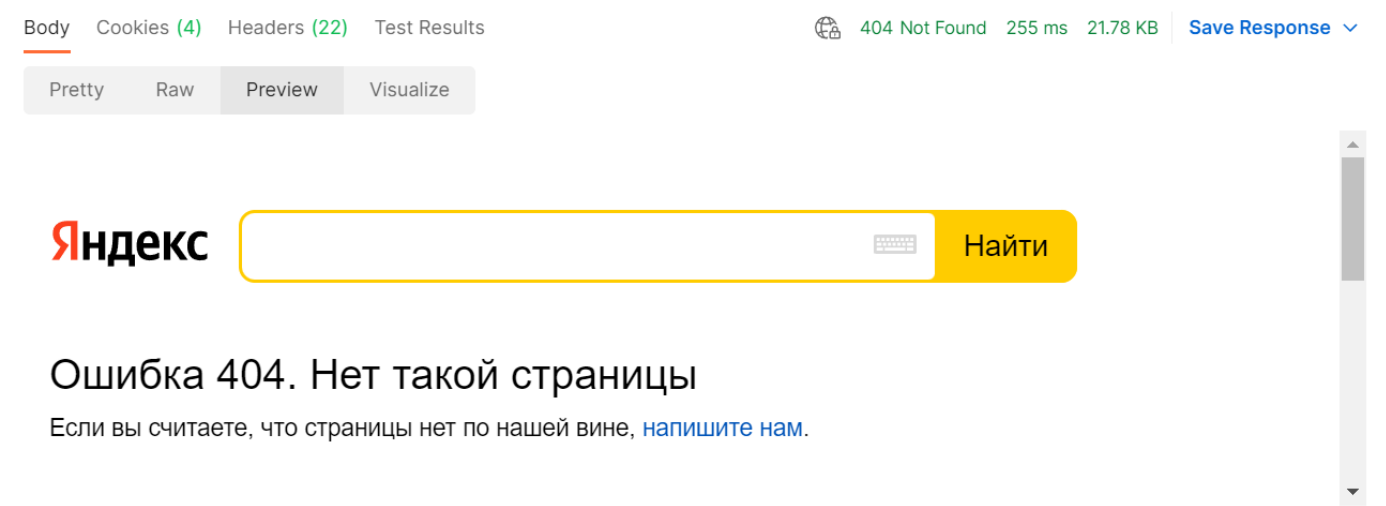
{"args":{"name":"anna","surname":"ivanova"},"headers":{"x-forwarded-proto":"https","x-forwarded-port":"443","host":"postman-echo.com","x-amzn-trace-id":"Root=1-62b4602b-4b6ceda21f93c2ed78e83c58","content-length":"276","user-agent":"PostmanRuntime/7.29.0","accept":"*/*","cache-control":"no-cache","postman-token":"84ae67c8-86d3-43ee-9a76-000f6a95554f","accept-encoding":"gzip, deflate, br","content-type":"multipart/form-data; boundary=-----694854588966882485506002","cookie":"sails.sid=s%3AKkyfMA_LGAZ4aegT_XuYxCh-Wf4AadNM.ARhHnzJ2PMFGVT0IaEs8hS8uDfGUiRGL7tVq1K%2BK%2BBE"},"json":null,"url":"https://postman-echo.com/post"}

```

Неформатированный (**Raw**) вид - текстовое представление тела ответа, без дополнительного форматирования. Позволяет посмотреть его в полученном виде.

Предпросмотр (**Preview**) - этот вид позволяет удобно просматривать полученное содержимое в обработанном виде. Это может быть удобно например в случае получения стандартных ошибок, которые приходят в виде **HTML** страницы.

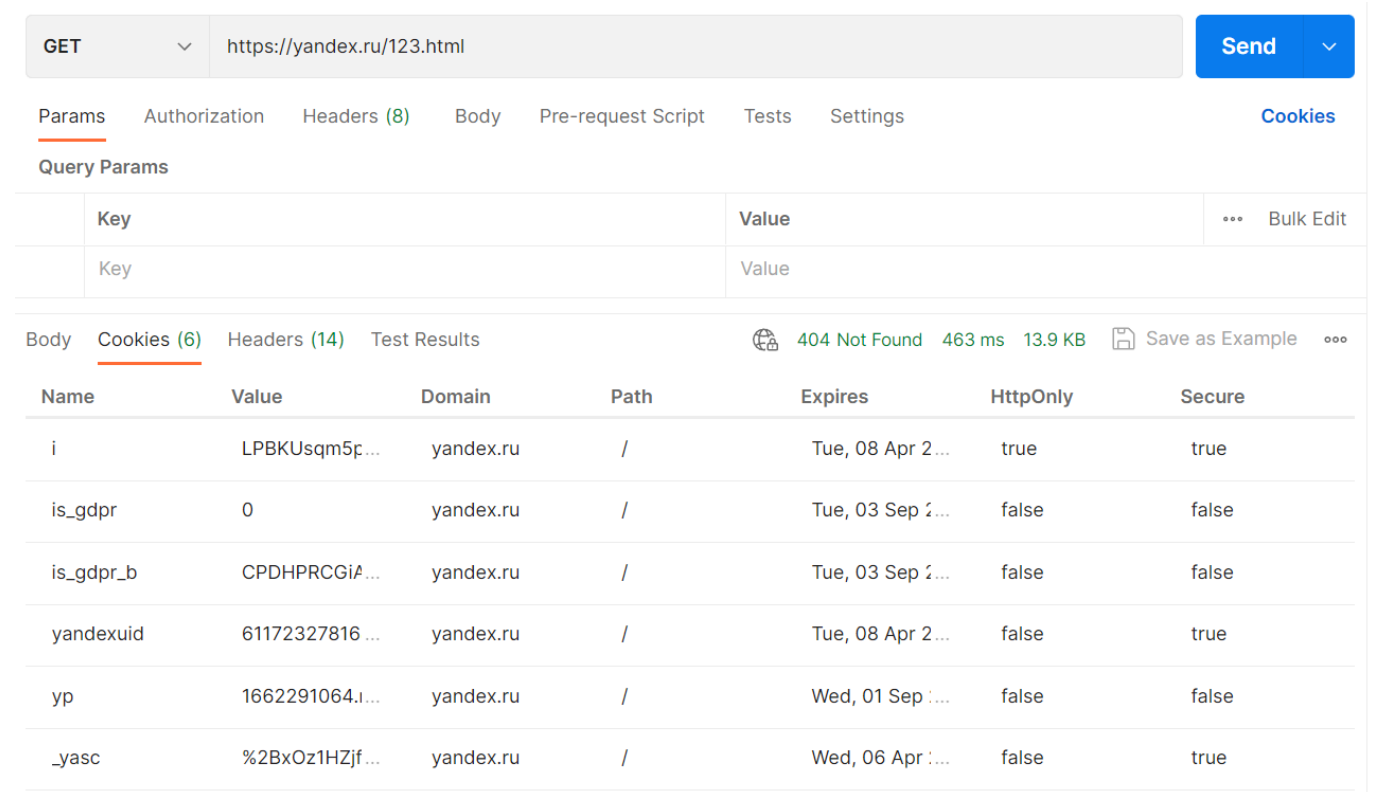
Для бинарных типов данных в ответах, таких как аудио файлы, **pdf** документы, **zip** архивы или что-то другое вы можете воспользоваться кнопкой **Send and download**, чтобы сохранить ответ и проверить его локально на своем компьютере.



Визуализация (**Visualize**) - позволяет обработать полученный ответ собственным способом. Код для визуализации добавляется отдельно.

Заголовки и cookies

В **Postman**, на вкладке **Cookies** можно просмотреть все куки отправленные сервером. Каждая запись будет содержать имя, значение, домен и путь, а также дополнительную информацию. В протоколе **HTTP Cookies** используются для хранения некоторой информации на стороне клиента. Она может быть использована для идентификации пользователей, сохранения настроек и предпочтений пользователей. Когда пользователь посещает веб-сайт, сервер отправляет на его компьютер специальный файл **cookie**, который затем сохраняется в браузере. Этот файл содержит информацию, которую сервер может использовать при последующих запросах от этого пользователя.



С помощью **cookies** веб-сайты могут например сохранять информацию о предпочтениях пользователя, таких как язык или настройки отображения. Предоставлять персонализированный контент, например определение региона пользователя или рекомендации товаров. Сохранять данные аутентификации пользователя, такие как логин и пароль, чтобы пользователь мог оставаться залогиненным на сайте. Следить за активностью пользователя на сайте, чтобы анализировать его поведение. Однако, в связи с тем, что **cookies** могут использоваться для отслеживания пользовательской активности, многие браузеры позволяют пользователям управлять их использованием, например, блокировать cookies от определенных сайтов или включать режим инкогнито.

На вкладке **headers** отображается информация о полученных заголовках, в формате ключ-значение. Если навести мышку на иконку информации, в поле с именем заголовка, можно получить информацию о каждом из них, в соответствии со спецификацией протокола **HTTP**.

Body Cookies (1) Headers (7) Test Results		200 OK 676 ms 1012 B Save Response	
KEY	VALUE		
Date	Thu, 23 Jun 2022 12:44:27 GMT		
Content-Type	application/json; charset=utf-8		
Content-Length	676		
Connection	keep-alive		
ETag	W/"2a4-RM6beOaSIDRBjIuBoLxNIhEESCY"		
Vary	Accept-Encoding		
set-cookie	sails.sid=s%3A6NsyrGT3SuwGFOz4xQHVPGSfLQZ4Xygj.k4qQIElf...		

Заголовки **HTTP** являются частью протокола **HTTP**. Они содержат метаданные о запросе или ответе, которые могут быть использованы для управления сеансом передачи данных, определения типа контента, управления кэшированием, аутентификации и многих других задач. Некоторые примеры использования заголовков **HTTP**:

- **Контроль кэширования.** Заголовки **HTTP** могут указывать, должны ли браузеры кэшировать ресурс и на какой период времени. Это позволяет уменьшить количество запросов к серверу и ускорить загрузку страниц.
- **Аутентификация.** **HTTP** заголовки могут использоваться для передачи информации об аутентификации, такой как токен доступа или логин и пароль. Это позволяет защитить ресурсы, доступ к которым должен быть ограничен.
- **Управление сеансом.** Здесь заголовки **HTTP** могут указывать, должны ли браузеры сохранять данные о сеансе и как долго они должны храниться. Это может быть использовано для создания персонализированных сеансов для каждого пользователя.
- **Определение типа контента.** В этом случае **HTTP** заголовки могут указывать тип контента, который возвращается сервером, например, **HTML**, изображение или документ **PDF**. Это помогает браузеру правильно отображать контент и обрабатывать его соответствующим образом.

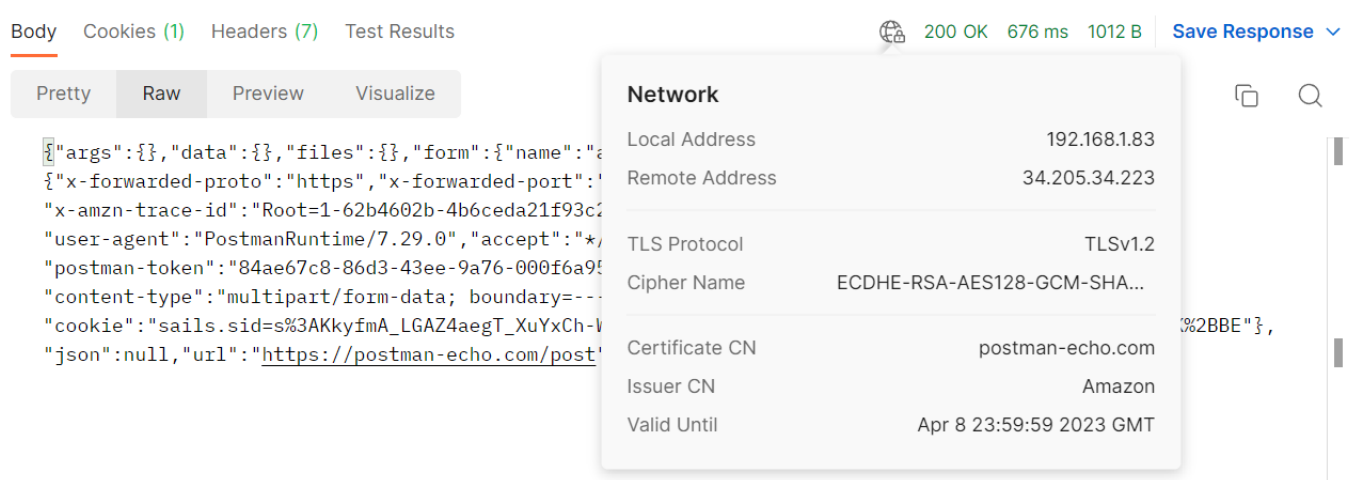
В целом, заголовки **HTTP** играют важную роль в управлении передачей данных между веб-серверами и веб-браузерами, позволяя контролировать поведение браузеров и обеспечивать более безопасную, эффективную и удобную работу с веб-ресурсами. Существует множество заголовков

HTTP, каждый из которых выполняет свою задачу. Вот некоторые из наиболее распространенных заголовков **HTTP**, которые используются веб-серверами и браузерами:

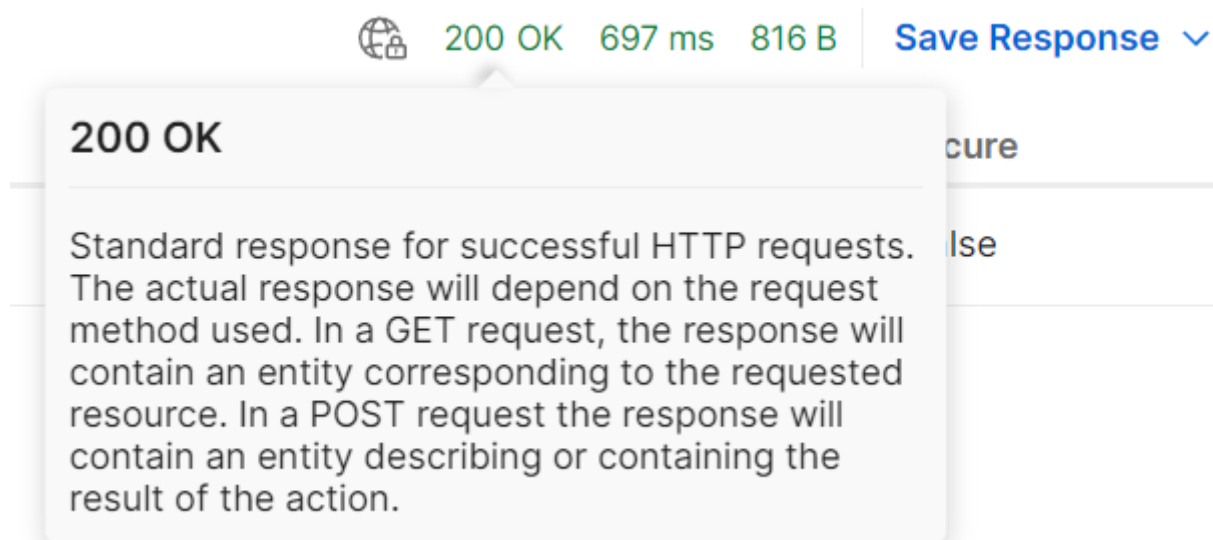
- **User-Agent** - позволяет серверу идентифицировать браузер или другое приложение, которое отправляет запрос на сервер.
- **Content-Type** - указывает тип контента, возвращаемого сервером, например, текст, **HTML**, **JSON**, изображение и т.д.
- **Content-Length** - указывает размер содержимого ответа в байтах.
- **Cache-Control** - позволяет определить, должен ли браузер кэшировать ответ и на какой период времени.
- **Accept** - позволяет клиенту указать типы контента, которые он готов принять от сервера.
- **Authorization** - используется для передачи информации об аутентификации, например, токен доступа или логин и пароль.
- **User-Cache-Control** - позволяет клиенту управлять кэшированием ответа на стороне браузера.

Это лишь некоторые из наиболее распространенных заголовков **HTTP**, которые могут использоваться при передаче данных между веб-серверами и веб-браузерами. Конкретные заголовки, используемые в каждом запросе или ответе, зависят от конкретных требований и настроек сервера и клиента. В случае отправки **HEAD** запроса, **Postman** откроет вкладку **Headers** по умолчанию, вместо вкладки **Body**.

Postman также отображает сетевую информацию при отправке запроса. Наведите мышку на иконку глобуса рядом со статус-кодом.



Далее идет код ответа, который вернул **API**. Наведите курсор на сам код для получения описания того, что он означает.



Код ответа **HTTP** — это числовой код, который возвращает сервер в ответ на **HTTP-запрос** от клиента. Код ответа состоит из трех цифр, где первая цифра указывает на класс ответа, а две последующие цифры — на конкретный код ответа внутри этого класса. Классы кодов ответа **HTTP** делятся на 5 групп:

- **Информационные (Info)** — начинаются с цифры **1**. Они сообщают клиенту, что запрос получен, и сервер продолжает обработку.
- **Успешные (Success)** — начинаются с цифры **2** и сообщают клиенту, что запрос выполнен успешно.
- **Перенаправления (Redirect)** — начинаются с цифры **3**. Сообщают клиенту, что дополнительные действия должны быть выполнены для завершения запроса.
- **Ошибки клиента (Client Error)** — начинаются с цифры **4** и сообщают клиенту, что запрос был сформулирован неправильно или не может быть выполнен.
- **Ошибки сервера (Server Error)** — начинаются с цифры **5**. Они сообщают клиенту, что запрос не может быть выполнен из-за ошибки на стороне сервера.

Коды ответа **HTTP** могут помочь в понимании, что произошло в процессе запроса и ответа на сервере. В дальнейшем это поможет правильно обработать ответ от сервера в своем приложении или браузере. Например, код **200** означает, что запрос был выполнен успешно, а код **404** означает, что запрошенный ресурс не найден на сервере. Чаще всего вы будете работать с успешными кодами ответа, а также с кодами ошибок клиента и сервера.

Затем следует время и размер ответа. **Postman** автоматически посчитает время ответа от сервера в миллисекундах. Эта информация может быть полезна, при анализе причин слишком долгих ответов. При наведении курсора на число, отображается разбиение этого времени по различным фазам ответа.

Далее **Postman** отображает примерный размер ответа. При наведении на число можно увидеть размер тела ответа и заголовков.

Body
Cookies (1)
Headers (7)
Test Results

200 OK
676 ms
1012 B
Save Response

Pretty
Raw
Preview
Visualize

```

{
  "args": {},
  "data": {},
  "files": {},
  "form": {
    "name": "anna",
    "surrogate": {
      "x-forwarded-proto": "https",
      "x-forwarded-port": "443",
      "host": "postman-echo.com",
      "x-amzn-trace-id": "Root=1-62b4602b-4b6cceda21f93c2ed78e83c58-62b4602b-4b6cceda21f93c2ed78e83c58",
      "user-agent": "PostmanRuntime/7.29.0",
      "accept": "*/*",
      "cache-control": "no-cache",
      "postman-token": "84ae67c8-86d3-43ee-9a76-000f6a95554f",
      "accept-encoding": "gzip, deflate, br",
      "content-type": "multipart/form-data; boundary=-----",
      "cookie": "sails.sid=s%3AKkyfmA_LGAZ4aegT_XuYxCh-Wf4AadNM.Af4aegT_XuYxCh-Wf4AadNM",
      "json": null,
      "url": "https://postman-echo.com/post"
    }
  }
}

```

EVENT	TIME
Prepare	142.98 ms
Socket Initialization	5.18 ms
DNS Lookup	138.34 ms
TCP Handshake	184.39 ms
SSL Handshake	142.53 ms
Transfer Start	175.37 ms
Download	29.93 ms
Process	0.44 ms
Total	819.13 ms

Еще в **Postman** есть функция сохранения ответов от сервера. Если запрос был сохранен в коллекцию, вы также можете сохранить и ответ для него. Для этого выберите **Save Response**, затем **Save as example**. Либо сохраните ответ в файл, выбрав **Save to a file**.